

MỤC LỤC

Trang

PHẦN I: ĐẶT VẤN ĐỀ.....	2
1. Lý do chọn đề tài.....	2
2. Mục đích nghiên cứu.....	3
3. Nhiệm vụ nghiên cứu của đề tài.....	3
4. Đối tượng nghiên cứu của đề tài.....	3
5. Phạm vi nghiên cứu của đề tài.....	3
PHẦN II: NỘI DUNG NGHIÊN CỨU	4
1. Cơ sở lý luận	4
1.1. Giới thiệu.....	4
1.1.1. Con trỏ là gì?.....	4
1.1.2. Làm thế nào để sử dụng thuật toán hai con trỏ?	5
1.2. Một số dạng về thuật toán hai con trỏ.....	6
1.2.1. Hai con trỏ, một con trỏ ở đầu và một con trỏ ở cuối di chuyển vào giữa cho đến khi cả 2 gặp nhau.....	6
1.2.2. Một con trỏ di chuyển chậm và một con trỏ di chuyển với tốc độ nhanh hơn.....	9
1.2.3. Hai con trỏ di chuyển trên hai mảng hoặc xâu	11
2. Cơ sở thực tiễn	15
2.1. Thực trạng của vấn đề trước khi áp dụng đề tài.....	15
2.1.1. Đặc điểm tình hình.....	15
2.1.2. Thực trạng trước khi nghiên cứu.....	16
2.1.3. Các giải pháp giải quyết vấn đề	17
2.2. So sánh cài đặt thuật toán 2 con trỏ và một số thuật toán khác.	17
2.3. Rèn luyện kỹ năng vận dụng thuật toán 2 con trỏ để giải một số bài toán cơ bản đến nâng cao.....	28
2.3.1. Một số bài tập về 2 con trỏ, một con trỏ ở đầu và một con trỏ ở cuối di chuyển vào giữa cho đến khi cả 2 gặp.	28
2.3.2. Một số bài tập về một con trỏ di chuyển chậm và một con trỏ di chuyển với tốc độ nhanh hơn.....	31
2.3.3. Hai con trỏ di chuyển trên hai mảng hoặc xâu	36
2.4. Bài tập tự giải có hướng dẫn.....	41
PHẦN III: KẾT LUẬN.....	48
1. Với mục tiêu đề ra đề tài đã làm được	48
2. Hướng phát triển của đề tài	48
3. Kiến nghị và đề xuất	48
TÀI LIỆU THAM KHẢO	50

PHẦN I: ĐẶT VẤN ĐỀ

1. Lý do chọn đề tài

Bộ trưởng Nguyễn Mạnh Hùng cũng phát biểu: “Mỗi người phải biết 3 ngôn ngữ: Tiếng mẹ đẻ để giữ gìn văn hóa truyền thống, Tiếng Anh để hội nhập quốc tế và ngôn ngữ máy lập trình coding để giao tiếp người với máy”.

Điều đó khẳng định vai trò và vị trí quan trọng của Tin học đối với toàn xã hội. Vì vậy mỗi người, mỗi học sinh cần hiểu và trang bị kiến thức cơ bản về Tin học để có thể theo kịp với thời đại, với sự phát triển của xã hội. Vì vậy khi học tin thì cần trang bị kiến thức, kỹ năng lập trình để giải quyết bài toán dễ dàng hơn.

Trong chương trình giáo dục phổ thông 2018 thì ngôn ngữ lập trình pascal không được đưa vào dạy học thay vào đó là ngôn ngữ lập trình Python. Ngoài Python thì C++ cũng là ngôn ngữ lập trình hiện nay rất phổ biến trong chương trình dạy học cũng như tính ứng dụng của 2 ngôn ngữ này rất nhiều, nhất là trong các kỳ thi tin học trẻ, thi vào chuyên tin, học sinh giỏi tỉnh...

Khi giải các bài toán Tin học người lập trình luôn mong muốn viết chương trình với thuật toán tối ưu, thời gian thực hiện nhanh, bộ nhớ hạn chế... Tuy nhiên, bài toán Tin học thường đa dạng, phong phú nên để có thể tìm được thuật toán tối ưu phù hợp dữ liệu bài toán là việc không hề dễ dàng. Trong lập trình tin học đã có rất nhiều phương pháp giải các bài toán nhưng để đảm bảo thời gian, không gian là không dễ. Vì vậy lựa chọn thuật toán để tối ưu là rất quan trọng.

Qua quá trình giảng dạy, học tập, tìm tòi và đặc biệt là tham gia bồi dưỡng học sinh giỏi nhiều năm qua, tôi đã tích lũy được một số kinh nghiệm về thuật toán 2 con trỏ. Do đó, tôi quyết định viết sáng kiến kinh nghiệm: “*Vận dụng thuật toán 2 con trỏ vào giải một số bài toán bồi dưỡng học sinh giỏi, thi vào chuyên phan trên ngôn ngữ lập trình C++ và Python*”.

Đề tài tập trung nghiên cứu về thuật toán 2 con trỏ, đưa ra những ứng dụng của 2 con trỏ khi giải các bài toán trên máy tính. Nhằm giúp học sinh vận dụng thuật toán này linh hoạt, giúp cải thiện về thời gian và không gian khi gặp các dạng bài toán này.

Để hoàn thành nhiệm vụ của đề tài, tôi đã nghiên cứu rất nhiều sách và các chuyên đề Tin học dành cho học sinh giỏi, các tài liệu trên các trang web. Tuy nhiên rất ít tài liệu trình bày cụ thể về cách sử dụng thuật toán này một cách đầy đủ và dễ hiểu.

2. Mục đích nghiên cứu

- Đề tài nêu ra các định hướng giúp học sinh có thể tiếp cận thuật toán 2 con trỏ để giải một số bài toán, tối ưu phù hợp với dữ liệu bài toán.

- Giúp học sinh tiếp cận ngôn ngữ lập trình C++ và Python sớm, tốt hơn.

- Từ đó bồi dưỡng học sinh năng lực giải quyết vấn đề trong giải toán Tin học, đồng thời rèn luyện và nâng cao kĩ năng lập trình cho các em. Đặc biệt là học sinh tham gia dự thi học sinh giỏi cấp tỉnh THCS, THPT hoặc thi vào các trường chuyên.

3. Nhiệm vụ nghiên cứu của đề tài

- Đề tài phân tích các thuật toán trong các dạng toán quen thuộc, so sánh độ phức tạp thuật toán và định hướng lựa chọn thuật toán tối ưu trong các trường hợp dữ liệu cụ thể nhằm giải bài toán hiệu quả nhất.

- Minh họa bằng các ví dụ, hình ảnh, video cụ thể. Đồng thời liên hệ các đề thi vào trường chuyên, đề thi học sinh giỏi tỉnh thời gian qua.

4. Đối tượng nghiên cứu của đề tài

- Độ phức tạp thuật toán và giải pháp lựa chọn thuật toán tối ưu trong các dạng bài toán quen thuộc trên ngôn ngữ lập trình C++ và Python.

- Phương pháp bồi dưỡng năng lực giải quyết vấn đề cho học sinh.

5. Phạm vi nghiên cứu của đề tài

- Chương trình Tin học THCS, THPT để bồi dưỡng học sinh giỏi Tin học và thi vào trường chuyên THPT.

PHẦN II: NỘI DUNG NGHIÊN CỨU

1. Cơ sở lý luận

Giới thiệu về thuật toán, các bước tiếp cận với thuật toán, khi nào thì ta sử dụng thuật toán hai con trỏ. Trình bày các dạng, phân tích và cài chương trình cụ thể để bạn đọc dễ hiểu nhất. Qua đó có thể vận dụng để giải quyết các bài toán về sau.

1.1. Giới thiệu

“Khi lựa chọn thuật toán dùng để giải quyết bài toán, thuật toán hiệu quả nhất là những thuật toán đơn giản và được thực thi tốt nhất”. Trong Cấu trúc dữ liệu và giải thuật thì thuật toán hai con trỏ là một thuật toán khá đơn giản và hiệu quả. Thường được ứng dụng để giải các bài toán trong lập trình, thuật toán này khá phổ biến đối với các kỳ thi tin học hiện nay.

Hai con trỏ là một dạng thuật toán trong đó hai con trỏ lặp lại trên cấu trúc dữ liệu cho đến khi một hoặc cả hai con đáp ứng điều kiện cần thiết.

Tuy nhiên; "*thuật toán hai con trỏ*" có một số khái niệm hạn chế. Hơn nữa, nó là một thuật toán đơn giản và hiệu quả, khi biết sử dụng đúng cách, nó sẽ mang lại rất nhiều hiệu quả.

Thuật toán hai con trỏ là một trong những bài thường gặp nhất trong bất kỳ cuộc thi lập trình. Cách tiếp cận này tối ưu hóa thời gian chạy bằng cách sử dụng một số thứ tự (không nhất thiết phải sắp xếp) của dữ liệu. Nó thường được áp dụng trên danh sách (mảng) và danh sách liên kết. Điều này thường được sử dụng để tìm kiếm các cặp trong một mảng được sắp xếp. Cách tiếp cận này hoạt động trong không gian không đổi. Mục đích chính của thuật toán này là giảm độ phức tạp của giải pháp dựa trên $O(n^3)$ hoặc $O(n^2)$ thành giải pháp thời gian tuyến tính.

Trong đề tài này, chúng tôi đã xem xét các khái niệm cơ bản và cung cấp các ví dụ khác nhau. Lấy các ví dụ minh họa cũng như một số bài tập rèn luyện tư duy và cách tiếp cận về thuật toán này từ cơ bản đến nâng cao giúp các em có thể va chạm các dạng bài tập một cách đa dạng hơn, để biết khi nào và làm thế nào để áp dụng thuật toán hai con trỏ.

Ngoài ra chúng tôi còn xây dựng các hình ảnh, video cụ thể để mô phỏng thuật toán. Qua đó người đọc dễ hiểu, dễ nắm bắt được phương pháp một cách tốt nhất.

1.1.1. Con trỏ là gì?

Một con trỏ là một tham chiếu đến một đối tượng. Đối tượng đó lưu trữ một địa chỉ bộ nhớ có giá trị khác nằm trong bộ nhớ máy tính, hoặc trong một số trường hợp, của phân cứng máy tính được ánh xạ bộ nhớ. Nói một cách đơn giản

hơn - một biến lưu trữ địa chỉ cho một mảng cũng là một con trỏ. Ví dụ, chúng tôi đã tính toán kích thước của các con trỏ trong chương trình C++ và Python.

Đặt câu hỏi thuật toán con trỏ và hai con trỏ có thể so sánh như thế nào?

Con trỏ lưu trữ địa chỉ của các đối tượng và chúng ta sẽ sử dụng thực tế này để trỏ đến các đối tượng khác nhau bằng cách sử dụng các biến trong thuật toán hai con trỏ. Do đó, trong trường hợp này, chúng cũng được gọi là con trỏ.

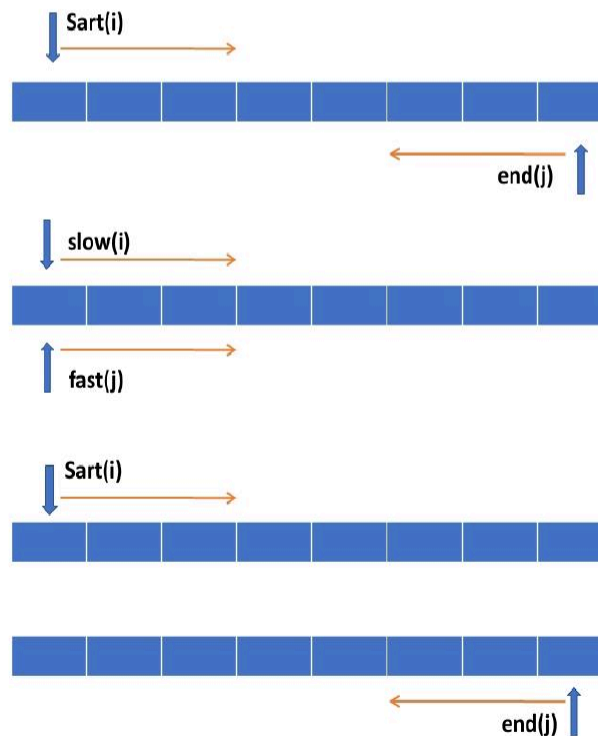
Thuật toán hai con trỏ không gì khác hơn là một sự tối ưu hóa của các kỹ thuật. Nó làm giảm sự phức tạp về thời gian bằng cách sử dụng một số loại thứ tự thay vì nhất thiết phải sắp xếp dữ liệu.

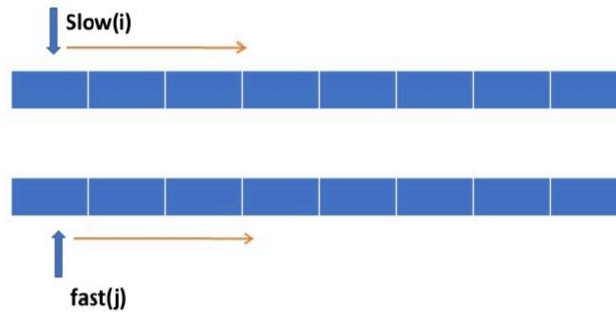
1.1.2. Làm thế nào để sử dụng thuật toán hai con trỏ?

Trước khi chúng ta bắt đầu, hãy nhớ rằng hai con trỏ trong thuật toán này đại diện cho hai chỉ số, số lần di chuyển của con trỏ này không phụ thuộc vào con trỏ kia.

Các bước trong cách tiếp cận hai con trỏ:

Một số hình ảnh của thuật toán 2 con trỏ.





Như trong hình trên, cách tiếp cận hai con trỏ có ba bước chính:

Khởi tạo con trỏ - Con trỏ có thể ở bất kỳ đâu tùy thuộc vào yêu cầu của bài toán chúng ta đặt vị trí xuất phát và kết thúc cho hợp lý. Chúng ta có thể sử dụng cả hai con trỏ bắt đầu ở cùng một vị trí tức là bắt đầu của mảng hoặc xâu, một con trỏ di chuyển chậm và con trỏ kia di chuyển nhanh hơn và có thể sử dụng một con trỏ ở vị trí bắt đầu và một con khác ở vị trí cuối cùng.

Chuyển động của con trỏ - Điều này sẽ quyết định cách chúng ta đưa ra cách tiếp cận bài toán, xác định giải pháp. Con trỏ có thể di chuyển theo cùng một hướng hoặc chúng có thể di chuyển theo hướng ngược lại. Mỗi lần di chuyển của các con trỏ là một đơn vị, cũng có trường hợp hai con trỏ này có thể đứng lệch nhau một đơn vị.

Điều kiện dừng - Điều này quyết định khi nào chúng ta dừng lại. Khi hai con trỏ gặp nhau hoặc một trong hai con trỏ di chuyển chạm điểm cuối của bên kia.

1.2. Một số dạng về thuật toán hai con trỏ.

1.2.1. Hai con trỏ, một con trỏ ở đầu và một con trỏ ở cuối di chuyển vào giữa cho đến khi cả 2 gặp nhau.

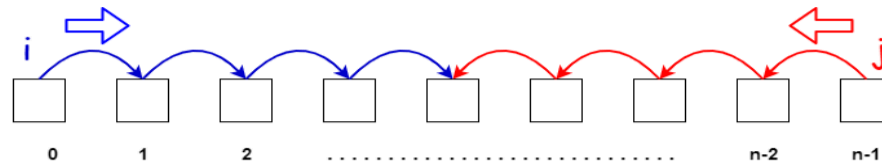
Trong ngôn ngữ lập trình thì ta thường làm quen với nhiều dạng bài tập khác nhau, nhất là một số dạng bài tập liên quan đến sắp xếp bạn có thể nghĩ đến ý tưởng hai con trỏ. Để bạn làm bài tập tốt hơn cũng như trong các kì thi bạn trang bị một số thuật toán để ứng dụng giải quyết bài toán tốt hơn. Dưới đây là một thuật toán hai con trỏ: một con trỏ ở đầu và một con trỏ ở cuối mảng hoặc chuỗi nào đó tùy thuộc vào bài toán, chọn di chuyển 2 con trỏ cho hợp lý theo yêu cầu bài toán. Thông thường dạng thuật toán này mảng sắp xếp ...thì thực hiện 3 bước sau:

Bước 1: Gán một con trỏ i đầu, con trỏ j cuối

Bước 2: Hai con trỏ di chuyển vào giữa

Bước 3: Điều kiện để hai con trỏ dừng khi chúng gặp nhau.

Hình ảnh minh họa:



Ví dụ: Cho một dãy số nguyên gồm N số hạng $a_1, a_2, a_3, \dots, a_N$. Biết $0 < N \leq 105$, $|a_i| \leq 109$. Viết chương trình đảo ngược các phần tử trong dãy số nguyên trên.

Dữ liệu vào từ tệp DAONGUOC.INP gồm:

- Dòng 1 Số N phần tử.
- Dòng 2 là dãy gồm N số từ $a_1, a_2, a_3, \dots, a_N$.

Dữ liệu ra. Ghi kết quả tệp DAONGUOC.OUT

Ví dụ:

DAONGUOC.INP	DAONGUOC.OUT
5	5 4 3 2 1
1 2 3 4 5	

Phân tích: Xét ví dụ sau.

Ta có dãy số: $a_1, a_2, a_3, \dots, a_{N-3}, a_{N-2}, a_{N-1}, a_N$

Dãy cần tìm là: $a_N, a_{N-1}, a_{N-2}, a_{N-3}, \dots, a_3, a_2, a_1$

Đối với bài toán này cách khá đơn giản chúng ta có thể duyệt ngược từ cuối mảng về phía đầu mảng và đưa ra các phần tử của mảng, chúng ta có thể duyệt từ đầu đến cuối hoặc nửa mảng và đưa các phần tử ra. Ngoài cách trên chúng tôi giới thiệu cách sử dụng thuật toán hai con trỏ.

Ta nhận thấy dãy cần đưa ra $a_N, a_{N-1}, a_{N-2}, a_{N-3}, \dots, a_3, a_2, a_1$ là dãy đảo ngược của dãy $a_1, a_2, a_3, \dots, a_{N-3}, a_{N-2}, a_{N-1}, a_N$. Vậy phần tử ở vị trí đầu tiên (a_1) và phần tử ở vị trí cuối cùng (a_N) của dãy a hoán đổi nhau, phần tử ở vị trí thứ 2 (a_2) và phần tử ở vị trí thứ $N-1$ (a_{N-1}) của dãy a hoán đổi nhau, tương tự có các cặp (a_3, a_{N-2}), (a_4, a_{N-3}), Như vậy ta chỉ cần hoán đổi các cặp đó cho nhau thì ta được kết quả bài toán. Ngoài ra ta thấy các cặp duyệt từ trái qua phải cứ mỗi cặp chỉ số tăng một đơn vị, duyệt từ phải qua trái mỗi cặp chỉ số giảm 1 đơn vị. Vì vậy ta sử dụng thuật toán hai con trỏ, một con trỏ di chuyển trái sang phải, con trỏ kia di chuyển ngược lại.

Cách tiếp cận: Cách tiếp cận sử dụng thuật toán hai con trỏ.

- Chúng tôi chỉ sử dụng hai con trỏ, một con trỏ ở đầu mảng và một con trỏ khác ở cuối mảng.

- Hoán đổi các phần tử khi con trỏ trở tới phần tử ở vị trí đó cho đến khi con trỏ đầu lớn hơn hoặc bằng con trỏ cuối thì kết thúc. Mỗi lần hoán đổi thì tăng con trỏ đầu và giảm con trỏ cuối một đơn vị

Giải pháp:

- Đầu tiên lấy hai con trỏ i và j .
- $i = 1$ và $j = N$.
- Bây giờ hãy chạy một vòng lặp while với điều kiện $i < j$. Bên trong vòng lặp while:
 - Hoán đổi $A[i]$ và $A[j]$.
 - Và sau khi hoán đổi, tăng i lên một và giảm dần j một
- Đưa ra kết quả bài toán

Link mô phỏng thuật toán: <https://youtu.be/WGz-uOsUWEo>

Độ phức tạp thời gian sẽ là: $O(N)$.

Code viết bằng ngôn ngữ lập trình C++	Code viết bằng ngôn ngữ lập trình Python
<pre>#include <bits/stdc++.h> #define ll long long using namespace std; ll a[100005];int n; int main(){ cin>>n; for (int i=1;i<=n;i++) cin>>a[i]; int i = 1; int j = n; while (i < j) { swap(a[i],a[j]); i++; j--; } for (int i=1;i<=n;i++) cout<<a[i]<<" "; return 0; }</pre>	<pre>def daonguoc(a): i, j = 0, len(a)-1 while i < j: a[i], a[j] = a[j], a[i] i += 1 j -= 1 m=int(input()) a=list(map(int,input().split())) daonguoc(a) print(a)</pre>

1.2.2. Một con trỏ di chuyển chậm và một con trỏ di chuyển với tốc độ nhanh hơn.

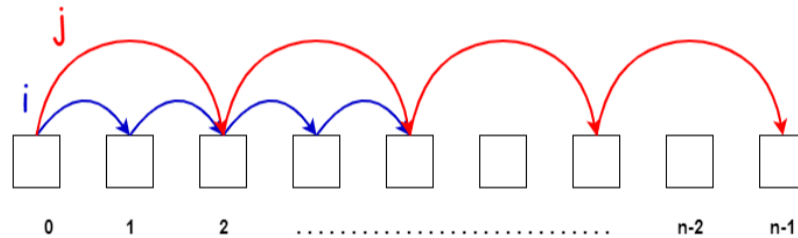
Đối với một số bài tập bạn thường gặp như dãy con liên tiếp, xâu con thì bạn có thể áp dụng hai con trỏ để giải quyết bài toán, chúng ta sử dụng hai con trỏ ở đầu mảng hoặc xâu nào đó tùy thuộc vào bài toán mà di chuyển hai con trỏ cho hợp lý đáp ứng yêu cầu bài toán, tuy nhiên dạng bài toán này thường sử dụng một con trỏ di chuyển nhanh hơn con trỏ kia, khi đáp ứng yêu cầu bài toán thì con trỏ kia tiến hành di chuyển để đáp ứng bài toán tối ưu hơn. Thông thường dạng thuật toán này thì thực hiện 3 bước sau:

Bước 1: Gán hai con trỏ i, j đầu

Bước 2: Hai con trỏ di chuyển về phía sau

Bước 3: Điều kiện để dừng một trong hai con trỏ di chuyển đến cuối

Hình ảnh minh họa:



Ví dụ: Xóa các ký tự trùng lặp liên tiếp khỏi một chuỗi.

Cho một chuỗi S hãy xóa các ký tự trùng lặp liên tiếp khỏi chuỗi. Nói cách khác loại bỏ các ký tự giống nhau liên tiếp, ngoại trừ một ký tự.

VD: $S = \text{"aaaAAbbcccbd"}$ kết quả "aAbcbcd" .

Phân tích: Cho xâu $S = \text{"aaaAAbbcccbd"}$ đưa ra kết quả $S = \text{"aAbcbcd"}$.

$S = \text{"aaaAAbbcccbd"}$

So sánh $s[i]$ và $s[j]$ để tìm ký tự khác nhau.

$S = \text{"aaaAAbbcccbd"}$; so sánh $S[0] = \text{'a'}$ với $S[1] = \text{'a'}$ bằng nhau

$S = \text{"aaaAAbbcccbd"}$; so sánh $S[0] = \text{'a'}$ với $S[2] = \text{'a'}$ bằng nhau

$S = \text{"aaaAAbbcccbd"}$; so sánh $S[0] = \text{'a'}$ với $S[3] = \text{'A'}$ khác nhau thay vị trí $S[1]$ bằng $S[3]$; ta được $S = \text{"aAAbbcccbd"}$;

$S = \text{"aAAbbcccbd"}$; so sánh $S[1] = \text{'A'}$ với $S[4] = \text{'A'}$ bằng nhau

$S = \text{"aAAbbcccbd"}$; so sánh $S[1] = \text{'A'}$ với $S[5] = \text{'b'}$ khác nhau thay vị trí $S[2]$ bằng $S[5]$ ta được $S = \text{"aAbbcccbd"}$.

S= “**aAb**AA**bccccbd**” ; so sánh S[2]= ‘b’ với S[6]= ‘b’ bằng nhau
 S= “**aAb**AA**bccccbd**” ; so sánh S[2]= ‘b’ với S[7]= ‘c’ khác nhau thay vị trí S[3] bằng S[7] ta được S= “**aAbc**AA**bccccbd**”
 S= “**aAbc**AA**bccccbd**” ; so sánh S[3]= ‘c’ với S[8]= ‘c’ bằng nhau
 S= “**aAbc**AA**bccccbd**” ; so sánh S[3]= ‘c’ với S[9]= ‘c’ bằng nhau
 S= “**aAbc**AA**bccccbd**” ; so sánh S[3]= ‘c’ với S[10]= ‘b’ khác nhau thay vị trí S[4] bằng S[10] ta được S= “**aAbcb**AA**bccccbd**”
 S= “**aAbcb**AA**bccccbd**” ; so sánh S[4]= ‘b’ với S[11]= ‘d’ khác nhau thay vị trí S[5] bằng S[11] ta được S= “**aAbcbd**AA**bccccbd**”. Kết quả S= “**aAbcbd**AA**bccccbd**”

Cách tiếp cận: Str=”aaaAA**bccccbd**”

Tôi thực hiện di chuyển các phần tử khác nhau rồi đưa về phía trước của xâu theo yêu cầu đề bài.

- Duyệt từ đầu đến cuối xâu
- Cần so sánh phần tử thứ i và phần tử thứ j
 - + Nếu 2 phần tử khác nhau thì tại vị trí thứ i+1 gán phần tử vị trí thứ j, di chuyển j một đơn vị, lấy vị trí thứ i+1 so sánh với ký tự vị trí thứ j.
 - + Nếu 2 phần tử giống nhau thì di chuyển j một đơn vị
- Kết quả là copy từ đầu xâu đến vị trí thứ i+1

Giải pháp:

Dựa vào những phân tích ta có giải pháp sử dụng **hai con trỏ** như sau.

- Tạo hai con trỏ i và j đều được khởi tạo bằng 0.
- Bây giờ chạy một vòng lặp cho đến khi j < độ dài của chuỗi đầu vào S và thực hiện các bước sau:
 - Khi ký tự thứ i bằng với ký tự thứ j thì tăng j lên 1 đơn vị
 - Nếu ký tự i không bằng ký tự j thì hãy thực hiện các bước sau:
 - Tăng i lên 1 đơn vị
 - Cập nhật ký tự ở vị trí thứ i với ký tự ở vị trí thứ j.
- Bây giờ chuỗi cần tìm cuối cùng của chúng ta sẽ là đoạn [0, i].

Độ phức tạp thời gian: O (N)

Link mô phỏng: <https://youtu.be/5DzUbORvp4U>

Code viết bằng ngôn ngữ lập trình C++	Code viết bằng ngôn ngữ lập trình Python
<pre> #include <bits/stdc++.h> using namespace std; string xoakitulap(string str) { int i=0,j=0; int n=str.size(); while(j<n) { if(str[i] !=str[j]) { i++; str[i] = str[j]; } j++; } return str.substr(0,i+1); } int main(){ string str; cin>>str; cout<<xoakitulap(str); } </pre>	<pre> def xoakitulap(str) : str= list(str) i=0 j=0 n=len(str) while(j<n): if(str[i] !=str[j]): i += 1 str[i] = str[j] j += 1 return "".join(str[0:i+1]) str= input() print(xoakitulap(str)) </pre>

1.2.3. Hai con trỏ di chuyển trên hai mảng hoặc xâu

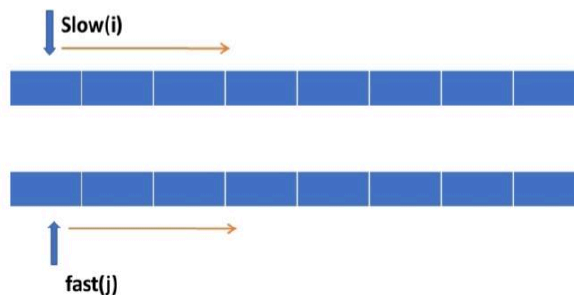
Đây là dạng bài tập dùng trên hai mảng, chúng ta có thể di chuyển một con trỏ ở đầu mảng thứ nhất và con trỏ kia ở cuối mảng thứ hai hoặc hai con trỏ ở đầu 2 mảng, một con di chuyển chậm và con trỏ kia di chuyển nhanh hơn tùy thuộc bài toán để chọn 1 trong 2 dạng dưới đây. Dạng thuật toán này thì thực hiện 3 bước sau:

Bước 1: Gán hai con trỏ i, j đầu

Bước 2: Hai con trỏ di chuyển về phía sau

Bước 3: Điều kiện để dừng một trong hai con trỏ di chuyển đến cuối

Hình ảnh minh họa hai con trỏ ở đầu hai mảng hoặc xâu:



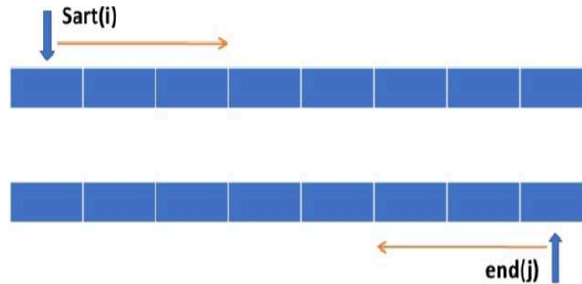
Hai con trỏ một con trỏ ở đầu, con trỏ kia ở cuối:

Bước 1: Gán một con trỏ i đầu mảng thứ nhất, còn con trỏ j cuối mảng thứ hai.

Bước 2: Hai con trỏ di chuyển vào giữa

Bước 3: Điều kiện để dừng hai con trỏ gặp nhau

Hình ảnh minh họa:



Ví dụ: Cho hai dãy số nguyên đã được **sắp xếp không giảm** a và b lần lượt có n và m phần tử. Hãy ghép chúng thành dãy c được bố trí theo thứ tự **không giảm**.

Giới hạn: $n \leq 10^5$, $m \leq 10^5$ và $0 \leq a_i, b_i \leq 10^9$, $0 \leq a_i, b_i \leq 10^9$.

Hãy cùng xem ví dụ sau đây.

Cho trước hai dãy số a và b được sắp xếp không giảm:

$a = [1, 3, 5, 9, 10]$

$b = [2, 4, 7, 13, 14, 16]$

Làm cách nào để có thể ghép chúng thành một dãy số c cũng được sắp xếp không giảm?

Trước tiên, hãy cùng xác định phần tử đầu tiên của dãy c .

Vì dãy c được bố trí theo thứ tự không giảm, cho nên phần tử đầu tiên của dãy c phải là phần tử có giá trị nhỏ nhất trong cả hai dãy a và b .

Ta có thể so sánh hai phần tử nhỏ nhất của hai dãy a, b và đưa phần tử có giá trị nhỏ hơn vào vị trí đầu tiên của dãy c .

Dãy a và b đã được sắp xếp không giảm, vì thế hai phần tử nhỏ nhất ở đây chính là hai phần tử ở vị trí đầu tiên ở mỗi dãy ($a[1]$ và $b[1]$).

So sánh $b[1]$ và $a[1]$, chọn phần tử có giá trị nhỏ hơn dãy và đưa vào dãy c .

a=[1, 3, 5, 9, 10]
b=[2, 4, 7, 13, 14, 16]
c=[1]

Bây giờ, phần tử tiếp theo của dãy c sẽ là phần tử nhỏ nhất trong các phần tử chưa được đưa vào dãy c.

Dãy a và b đã được sắp xếp không giảm, vì thế sau khi đưa a[1] vào dãy c, a[2] là phần tử nhỏ nhất chưa được chọn ở dãy a và b[1] là phần tử nhỏ nhất chưa được chọn ở dãy b.

So sánh a[2] và b[1], chọn phần tử có giá trị nhỏ hơn và đưa vào dãy c.

a=[1, 3, 5, 9, 10]
b=[2, 4, 7, 13, 14, 16]
c=[1, 2]

Sau khi đưa b[1] vào dãy c, b[2] trở thành phần tử nhỏ nhất chưa được chọn ở dãy b.

Vẫn như thế, phần tử tiếp theo của dãy c sẽ là phần tử nhỏ nhất trong các phần tử chưa được đưa vào dãy c.

So sánh b[2] và a[2], chọn phần tử có giá trị nhỏ hơn dãy và đưa vào dãy c.

a=[1, 3, 5, 9, 10]
b=[2, 4, 7, 13, 14, 16]
c=[1, 2, 3]

Sau khi đưa a[2] vào dãy c, a[3] trở thành phần tử nhỏ nhất chưa được chọn ở dãy a.

So sánh b[2] và a[3], chọn phần tử có giá trị nhỏ hơn dãy và đưa vào dãy c.

a=[1, 3, 5, 9, 10]
b=[2, 4, 7, 13, 14, 16]
c=[1, 2, 3, 4]

Tương tự ta thu được dãy c=[1, 2, 3, 4, 5, 7, 9, 10, 13, 14, 16]

Cách tiếp cận:

- Tại mọi thời điểm, phần tử tiếp theo được đưa vào dãy c sẽ là phần tử có giá trị nhỏ nhất trong các phần tử chưa được chọn.
- * Bằng cách so sánh phần tử nhỏ nhất chưa được chọn ở dãy a và phần tử nhỏ nhất chưa được chọn ở dãy b, phần tử nhỏ hơn sẽ được chọn vào dãy c.
- Ban đầu, lúc dãy c chưa có phần tử nào
- * $a[1]$ là phần tử nhỏ nhất chưa được chọn trong dãy a.
- * $b[1]$ là phần tử nhỏ nhất chưa được chọn trong dãy b.
- Khi đưa phần tử $a[i]$ vào dãy c thì phần tử nhỏ nhất chưa được chọn trong dãy a sẽ là $a[i+1]$.
- Khi đưa phần tử $b[j]$ vào dãy c thì phần tử nhỏ nhất chưa được chọn trong dãy b sẽ là $b[j+1]$.

Giải pháp:

Dựa vào những phân tích ta có giải pháp sử dụng **hai con trỏ** như sau:

- Dãy a có con trỏ i, con trỏ này bắt đầu ở vị trí đầu dãy a.
 - Con trỏ i này được thể hiện như phần tử nhỏ nhất chưa được chọn trong dãy a.
- Dãy b có con trỏ j, con trỏ này bắt đầu ở vị trí đầu dãy b.
 - Con trỏ j này được thể hiện như phần tử nhỏ nhất chưa được chọn trong dãy b.
- Ta sẽ lặp lại công việc này, cho đến khi đưa hết các phần tử trong dãy a và b vào dãy c.
 - Khi các phần tử trong một dãy nào đó, dãy a hoặc dãy b, đều đã được đưa vào dãy c, đưa lần lượt các phần tử trong dãy còn lại vào dãy c.
 - Ngược lại:
 - + So sánh hai phần tử ở hai con trỏ.
 - + Đưa phần tử có giá trị nhỏ hơn vào dãy c, nếu hai phần tử có giá trị như nhau thì chọn một trong hai.
 - + Tăng vị trí con trỏ ở phần tử được đưa vào lên một đơn vị.

Độ phức tạp thời gian sẽ là: $O(n+m)$.

Link mô phỏng: <https://youtu.be/GaeZ4bexOeg>

Code viết bằng ngôn ngữ lập trình c++	Code viết bằng ngôn ngữ lập trình Python
<pre> #include <bits/stdc++.h> using namespace std; long long a[100005],b[100005];int n,m; int main(){ vector<int> c; cin>>n>>m; for (int i=0;i<n;i++) cin>>a[i]; for (int i=0;i<m;i++) cin>>b[i]; int i = 0, j = 0; while(i < n && j < m){ if(a[i] <= b[j]){ c.push_back(a[i]); ++i; } else{ c.push_back(b[j]); ++j; } } while(i < n){ c.push_back(a[i]);i++; } while(j < m){ c.push_back(b[j]);j++; } for (auto x:c) cout<<x<<" "; } </pre>	<pre> n,m=map(int,input().split()) a=list(map(int,input().split())) b=list(map(int,input().split())) i,j=0,0 c=[] while(i < n and j < m): if(a[i] <= b[j]): c.append(a[i]) i=i+1 else: c.append(b[j]) j=j+1 while(i < n): c.append(a[i]) i=i+1 while(j < m): c.append(b[j]) j=j+1 print(c) </pre>

2. Cơ sở thực tiễn

Nêu ra thực trạng vấn đề, những hạn chế, khó khăn của học sinh cũng như giáo viên trong việc giải quyết các bài toán lớn, các bài toán ôn thi học sinh giỏi hiện nay. Qua đó giải quyết vấn đề và đưa ra các bài toán điển hình để so sánh sử dụng thuật toán con trỏ và một số phương pháp khác. Sử dụng thuật toán 2 con trỏ có thể vận dụng lập trình giải các bài toán trong các đề thi hay khi ôn luyện.

Trang bị kiến thức cho học sinh và giáo viên để giải quyết tốt khi gặp các bài toán có liên quan đến sử dụng thuật toán 2 con trỏ. Sử dụng thuật toán 2 con trỏ một cách hiệu quả nhất.

2.1. Thực trạng của vấn đề trước khi áp dụng đề tài

2.1.1. Đặc điểm tình hình

*** Thuận lợi:**

- Với sự bùng nổ của ngành công nghệ thông tin cũng như chương trình giáo dục phổ thông 2018 các em học sinh, phụ huynh ngày càng quan tâm hơn.

- Ngày nay máy tính ngày càng có tốc độ xử lý cao đáp ứng được yêu cầu xử lý các bài toán có dữ liệu lớn trong thời gian thực hiện ngắn.
- Các cuộc thi lập trình này ngày càng tổ chức nhiều hơn, việc học, xem tài liệu, luyện làm code cũng dễ dàng hơn, có thể xem các tài liệu trên web cũng như các kênh như youtube....

*** Khó khăn:**

- Do môn Tin học chưa có trong tổ hợp môn thi tốt nghiệp và đại học nên các em và phụ huynh chưa quan tâm đến nên việc chọn học sinh giỏi cũng như các em chưa thực sự đầu tư chuyên sâu vào lập trình.
- Ngôn ngữ lập trình C++ và Python còn khá mới mẻ, những kiến thức trong chương trình Tin học phổ thông còn hạn chế, hoặc chưa đủ đáp ứng cho việc giải một số bài toán trong các kỳ thi học sinh giỏi Tỉnh khi có yêu cầu dữ liệu lớn cùng thời gian thực hiện ngắn.
- Các tài liệu tổng hợp các cách để giải quyết các bài toán yêu cầu cao như vậy chưa có nhiều để học sinh tham khảo, ôn luyện.

2.1.2. Thực trạng trước khi nghiên cứu

Ngày nay công nghệ thông tin là một trong những ngành hót, nhân lực còn thiếu. Mỗi năm sinh viên ra trường chưa đáp ứng đủ yêu cầu thị trường. Mức thu nhập ngành công nghệ thông tin là khá cao, nên việc quan tâm của các em ngày càng nhiều, nhất là trong các cuộc thi lập trình.

Với sự phát triển nhanh về tốc độ cũng như sự quan tâm đến của ngành công nghệ thông tin ngày càng cao thì đề thi học sinh giỏi Tin học cũng đòi hỏi ngày càng nâng cao hơn. Trong các đề thi học sinh giỏi thì người ta quan tâm về thời gian thực hiện và độ lớn của dữ liệu đầu vào. Đa số giáo viên gặp khó khăn trong việc hướng dẫn học sinh giải bài toán thế nào để đạt được trọn vẹn yêu cầu của bài toán. Những năm trước đây khi bồi dưỡng học sinh thi học sinh giỏi Tỉnh giáo viên cũng đã chú trọng nhiều hơn đến cải tiến chương trình tối ưu, tuy nhiên hiệu quả mang lại chưa cao.

Nguyên nhân chủ yếu của thực trạng này là giáo viên chưa có phương pháp giảng dạy phù hợp với vấn đề này. Cụ thể giáo viên thường giúp học sinh cải tiến thuật toán của từng bài cụ thể mà chưa phân loại thành các dạng bài, định hướng học sinh đánh giá thuật toán và cải thiện thuật toán nhưng chưa ước lượng với mỗi mức dữ liệu thì cần phải xây dựng thuật toán có độ phức tạp tương ứng như thế nào để có thể đáp ứng được... Cho nên học sinh sẽ gặp khó khăn trong việc vận dụng vào các bài toán tương tự, cũng như không linh hoạt trong các bài toán mới.

2.1.3. Các giải pháp giải quyết vấn đề

Có những bài toán nhiều khi có vẻ đơn giản, đa số học sinh có thể giải được nhưng khi thực hiện với dữ liệu lớn thì không đáp ứng được thời gian yêu cầu hoặc không thể thực hiện được tất cả các test yêu cầu. Lúc này đòi hỏi phải tìm được thuật toán tối ưu nhất.

Tối ưu hoá là một đòi hỏi thường xuyên trong quá trình giải quyết các bài toán tin học. Tối ưu hoá thuật toán là một công việc yêu cầu tư duy thuật toán rất cao, cùng với khả năng sử dụng thuần thục các cấu trúc dữ liệu. Vì vậy, việc tìm ra thuật toán tối ưu là không dễ chút nào. Tối ưu hoá thường được tiến hành theo 2 góc độ đó là tối ưu theo không gian có nghĩa là tối ưu không gian lưu trữ (bộ nhớ) và tối ưu theo thời gian có nghĩa là giảm độ phức tạp thuật toán, giảm các bước xử lý trong chương trình... Đa số giáo viên gặp khó khăn trong việc hướng dẫn học sinh thiết kế thuật toán hoặc lựa chọn thuật toán nào để có thể giảm độ phức tạp của thuật toán, đồng thời phù hợp với dữ liệu và yêu cầu của đề bài. Đa số khi chấm bài, dùng test chấm mới biết được chương trình đáp ứng được bao nhiêu test so với yêu cầu của đề ra. Điều này chỉ thực hiện được khi ôn luyện cho các em, còn khi các em đi thi thì không tự ước lượng được bài làm đạt kết quả thế nào.

Có một cách để ước lượng được chương trình giải được có thể đáp ứng được yêu cầu của đề hay không (nghĩa là đáp ứng được khoảng dữ liệu vào bao nhiêu), đó là đánh giá độ phức tạp của thuật toán trong chương trình. Như vậy để có thể tối ưu hóa thuật toán trước tiên chúng ta phải hiểu rõ về độ phức tạp của thuật toán và cách đánh giá ước lượng thuật toán so với dữ liệu đầu vào yêu cầu.

Thuật toán hai con trỏ không gì khác hơn là một sự tối ưu hóa của các kỹ thuật. Nó làm giảm sự phức tạp về thời gian bằng cách sử dụng một số loại thứ tự thay vì nhất thiết phải sắp xếp dữ liệu. Qua đó giúp chương trình chạy nhanh hơn, không gian lưu trữ ít hơn.

2.2. So sánh cài đặt thuật toán 2 con trỏ và một số thuật toán khác.

Khi bạn đứng trước bài toán bạn có thể có nhiều cách giải, nhưng việc lựa chọn thuật toán tối ưu mới là quan trọng, hiện nay trong các kì thi học sinh giỏi thường đề yêu cầu về thời gian. Vì vậy bạn phải lựa chọn thuật toán tối ưu nhất để đạt kết quả cao trong các kì thi, dưới đây chúng tôi dưới thiệu một số thuật toán ta thường gặp để so sánh với thuật toán 2 con trỏ.

Ví dụ 1: Cho một mảng A gồm N số nguyên đã sắp xếp (được sắp xếp theo thứ tự tăng dần). Hãy tìm xem có tồn tại cặp phần tử nào ($A[i]$, $A[j]$) khác nhau sao cho tổng của chúng bằng X.

Giới hạn: $2 \leq n \leq 10^6$ và $0 \leq a_i, X \leq 10^9$

Cho trước mảng số a được sắp xếp tăng dần và $X=16$.

$a = [2, 5, 6, 8, 10, 12, 15]$

Nếu tồn tại cặp có tổng =16 thì đưa ra thông báo "Yes" ngược lại không đưa ra là "No"

Hướng giải quyết 1: Sử dụng 2 vòng lặp for (Brute Force)

Độ phức tạp thời gian sẽ là: $O(n^2)$.

Brute Force là một thuật toán vét cạn, thuật toán này sẽ duyệt tất cả các trường hợp có thể có để giải quyết một vấn đề nào đó (Bao gồm cả trường hợp đúng và các trường hợp sai. Riêng với bài toán của chúng ta, thì thuật toán được đưa ra như sau:

- Duyệt 2 vòng for lồng nhau các phần tử có trong dãy số để có được 2 số a_i , a_j
- Kiểm tra xem, nếu $a_i + a_j = X$ và $i \neq j$ thì a_i, a_j là 2 số được chọn

Code viết bằng ngôn ngữ lập trình C++	Code viết bằng ngôn ngữ lập trình Python
<pre>#include <bits/stdc++.h> using namespace std; int A[100000];int N,X; bool isPairSum(int A[],int N,int X) { for (int i = 0; i < N; i++) { for (int j =i+1; j < N; j++) { if (A[i] + A[j] == X) return true; if (A[i] + A[j] > X) break; } } return false; } int main() { cin>>N>>X; for (int i=0;i<N;i++) cin>>A[i]; bool m=isPairSum(A,N,X); if(m) cout<<"ton tai 1 cap tong ="<<X; else cout<<"khong ton tai cap tong="<<X; return 0; }</pre>	<pre>def isPairSum(A, N, X): for i in range(0,N): for j in range(i+1,N): if (A[i] + A[j] == X): return True if (A[i] + A[j] > X): break return 0 m=int(input()) x=int(input()) arr=list(map(int,input().split())) x=isPairSum(arr, len(arr), x) if (x==True):print('có tồn tại 1 cap') else:print('không tồn tại 1 cap')</pre>

Hướng giải quyết 2: Tìm kiếm nhị phân - Binary Search

Độ phức tạp thời gian sẽ là: $O(n \log n)$.

Thuật toán tìm kiếm nhị phân hoạt động trên các mảng đã được sắp xếp. Hoạt động theo các trình tự:

- So sánh một phần tử đứng chính giữa mảng với giá trị cần tìm.
- Nếu bằng nhau, vị trí của nó trong mảng sẽ được trả về. Nếu giá trị cần tìm nhỏ hơn phần tử này, quá trình tìm kiếm tiếp tục ở nửa nhỏ hơn của mảng.

- Nếu giá trị cần tìm lớn hơn phần tử ở giữa, quá trình tìm kiếm tiếp tục ở nửa lớn hơn của mảng. Bằng cách này, ở mỗi phép lặp, thuật toán có thể loại bỏ nửa mảng mà giá trị cần tìm chắc chắn không xuất hiện

Code viết bằng ngôn ngữ lập trình C++

```
#include <bits/stdc++.h>
using namespace std;
int i,n,t,m,a[100005];
int binary(int a[],int l,int r,int ti)
{
    if (l > r) return -1;
    int mid = (l+r) / 2;
    if (a[mid] == ti) return 0;
    if (a[mid]>ti)
        return binary(a,l,mid-1,ti);
    return binary(a,mid+1,r,ti);
}
int main()
{
    cin>>n>>t;int ok=0;
    for (i=0;i<n;i++) cin>>a[i];
    for (int i = 0; i <n; i++) {
        int position=binary(a,0,n-1,t-a[i]);
        if (position !=-1 && position != i)
            { ok=1;break;}
    }
    if(ok==1)
        cout<<"ton tai tong bang:"<<t;
    else
        cout<<"khong ton tai tong bang:"<<t;
    return 0;
}
```

Code viết bằng ngôn ngữ lập trình Python

```
def binarySearch(left,right,target):
    if (left > right): return -1;
    mid = (left+right+1)// 2;
    if (a[mid] == target): return 1
    if (a[mid] > target):
        return binarySearch(left,mid-1,target);
    return binarySearch(mid + 1,right,target);
n=int(input())
x=int(input())
a=list(map(int,input().split()))
ok=2
for i in range(0,n-1):
    position = binarySearch(0, n-1, x-a[i]);
    if (position != -1 and position != i):
        ok=1
        break
if (ok==1):
    print("ton tai cap tong bang:",x)
else:
    print("khong tai cap tong bang:",x)
```

Hướng giải quyết 3: Tìm kiếm bằng 2 con trỏ - Two Pointe

Độ phức tạp thời gian sẽ là: $O(n)$.

Phân tích:

Trước tiên, ta có một chút nhận xét sau:

- $a[1] < a[2] < a[3] < a[4] < a[5] < a[6] < a[7]$ vì dãy a tăng dần.
- $a[1] + a[7] = 17 > X \Rightarrow X < a[1] + a[7] < a[2] + a[7] < a[3] + a[7] < a[4] + a[7] < a[5] + a[7] < a[6] + a[7]$.

Có thể thấy, tổng của $a[7]$ với các phần tử khác trong dãy đều lớn hơn X . Vì thế ta không quan tâm đến $a[7]$ nữa.

$a = [2, 5, 6, 8, 10, 12, 15]$

- $a[1]+a[6]=14 < X \Rightarrow a[1]+a[2] < a[1]+a[3] < a[1]+a[4] < a[1]+a[5] < a[1]+a[6] < X$.

Có thể thấy, tổng của $a[1]$ với các phần tử khác trong các phần tử ta quan tâm đều nhỏ hơn X . Vì thế ta không quan tâm đến $a[1]$ nữa.

$a=[2, 5, 6, 8, 10, 12, 15]$

- $a[2]+a[6]=17 > X \Rightarrow X < a[2]+a[6] < a[3]+a[6] < a[4]+a[6] < a[5]+a[6]$.

Có thể thấy, tổng của $a[6]$ với các phần tử khác trong các phần tử ta quan tâm đều lớn hơn X . Vì thế ta không quan tâm đến $a[6]$ nữa.

$a=[2, 5, 6, 8, 10, 12, 15]$

- $a[2]+a[5]=15 < 16 \Rightarrow a[2]+a[5] < a[3]+a[5] < a[4]+a[5]$ tương tự ta không cần quan tâm đến $a[2]$ nữa.

$a=[2, 5, 6, 8, 10, 12, 15]$ ta có $a[3]+a[5]=16=X$ là kết quả bài toán

Như vậy, tại một thời điểm bất kỳ, những phần tử chúng ta cần quan tâm đến sẽ là các phần tử trong đoạn $[i, j]$ nào đó.

Ý tưởng:

- Nếu $i=j$, trong mảng A không tồn tại hai vị trí khác nhau mà tổng hai phần tử ở đó có giá trị là X .

- Ngược lại:

- Nếu $a[i]+a[j]=X$, ta đã tìm được hai vị trí cần tìm (i và j).

- Nếu $a[i]+a[j] < X$, không quan tâm đến $a[i]$ nữa và các phần tử chúng ta cần quan tâm đó là các phần tử trong đoạn $[i+1, j]$.

- Nếu $a[i]+a[j] > X$, không quan tâm đến $a[j]$ nữa và các phần tử chúng ta cần quan tâm đó là các phần tử trong đoạn $[i, j-1]$.

Giải pháp:

Từ những phân tích vừa rồi ta có giải pháp sử dụng hai con trỏ như sau:

- Một con trỏ (i) được đặt ở đầu mảng A , con trỏ còn lại (j) được đặt ở cuối mảng A .

- Nếu tổng của hai phần tử ở hai vị trí con trỏ

- Nhỏ hơn X : tăng vị trí con trỏ i lên một đơn vị.

- Lớn hơn X : giảm vị trí con trỏ j đi một đơn vị.

- Tiếp tục di chuyển cho đến khi hai con trỏ gặp nhau.

- Khi con trỏ chưa gặp nhau mà tổng ở hai vị trí con trỏ có giá trị là X thì ta đã tìm được hai vị trí cần tìm (i và j), kết thúc chương trình.

Link mô phỏng thuật toán: <https://youtu.be/39F4jKIn1OI>

Code viết bằng ngôn ngữ lập trình C++

```

#include <bits/stdc++.h>
using namespace std;
int n,m;int b[100005];
bool isSum(int A[],int N,int X)
{
    int i = 0;
    int j = N - 1;
    while (i < j) {
        if (A[i] + A[j] == X)
            return 1;
        else if (A[i]+A[j]<X)
            i++;
        else
            j--;
    }
    return 0;
}
int main()
{
    cin>>n>>m;
    for (int i=0;i<n;i++) cin>>b[i];
    bool ok=isSum(b,n,m );
    if (ok)cout<<"ton tai tong bang:"<<m;
    else
        cout<<"khong ton tai tong bang:"<<m;
    return 0;
}

```

Code viết bằng ngôn ngữ lập trình Python

```

def isPairSum(A, N, X):
    i = 0
    j = N - 1
    while(i < j):
        if (A[i] + A[j] == X):
            return True
        elif(A[i] + A[j] < X):
            i += 1
        else:
            j -= 1
    return 0
m=int(input())
x=int(input())
arr=list(map(int,input().split()))

if (isPairSum(arr, len(arr), x)):
    print("tai cap tong bang:",x)
else:
    print("khong tai cap tong bang:",x)

```

Nhận xét: Qua ba hướng giải quyết trên thì hướng giải quyết 1 có độ phức tạp là $O(n^2)$ không đáp ứng được thời gian khi $n > 10^4$, hướng giải quyết 2 độ phức tạp $O(n \log n)$ và hướng giải quyết 3 có độ phức tạp $O(n)$. So sánh cả 3 hướng giải quyết trên thì hướng giải quyết 3 tốt nhất.

Ví dụ 2:

Cho một dãy số nguyên dương $a_1, a_2, \dots, a_N$ ($1 < N < 2 \cdot 10^5$), $a_i \leq 10^6$ với mọi $i = 1..N$ và một số nguyên dương K ($K < 10^9$).

Yêu cầu: Tìm độ dài nhỏ nhất của dãy con chứa các phần tử liên tiếp của dãy mà có tổng các phần tử lớn hơn hoặc bằng K .

Dữ liệu vào: Đọc từ file **SUB.INP** gồm 2 dòng, dòng 1 chứa N và K . Dòng 2 chứa các phần tử của dãy.

Dữ liệu ra: Kết quả ghi vào file **SUB.OUT**, là độ dài của dãy con tìm được.

Ví dụ:

SUB.INP	SUB.OUT
10 17 5 1 3 5 10 7 4 9 2 8	2

Hướng giải quyết 1. Sử dụng 2 vòng lặp for

Độ phức tạp thuật toán: $O(n^2)$

Ý tưởng: Sử dụng tổng tiền tố ta có $T[i] = a_1 + a_2 + a_3 + \dots + a_{i-1} + a_i = T[i-1] + a[i]$.

- Tính tổng tiền tố: $T[i] = T[i-1] + a[i]$
- Cho 2 vòng lặp for lồng nhau
Giá trị biến i chạy 1 đến n, biến j từ i đến n
Nếu $T[j] - T[i] > K$ nếu $smin = \min(smin, j-i)$.
- Đưa giá trị smin ra

Code viết bằng ngôn ngữ lập trình C++

```
#include<bits/stdc++.h>
using namespace std;
const int N=1e5+2;
int n,a[N],T[N],res,k;
int main(){
    cin>>n>>k;
    for (int i=1;i<=n;i++) cin>>a[i];
    T[0]=0;res=1e9;
    for(int i=1;i<=n;i++)T[i]=T[i-1]+a[i];
    for (int i=1;i<=n;i++)
    for (int j=i;j<=n;j++){
        if(T[j]-T[i]>=k) res=min(res,j-i);}
    cout<<res;
    return 0;
}
```

Code viết bằng ngôn ngữ lập trình Python

```
n,k=map(int,input().split())
a=list(map(int,input().split()))
res=1e9
T=[sum(a[: i+1])for i in range(len(a))]
for i in range(0,len(T)-1):
    for j in range(i,len(T)-1):
        if (T[j]-T[i]>=k):
            res=min(res,j-i)
print(res)
```

Hướng giải quyết 2: Tìm kiếm nhị phân - Binary Search

Độ phức tạp thuật toán: $O(n \log n)$

Ý tưởng:

Gọi $T[i]$ là tổng của các số từ $A[1]$ đến $A[i]$ vì $A[i]$ các số dương nên dãy T là dãy tăng dần Khi đó ta sẽ tiến hành tìm kiếm nhị phân trên dãy T như sau:

Xét $T[i]$ $d=1$; $c=i-1$; $g=(d+c)/2$

- Nếu $T[i] - T[g] > K$ thì $kq = \min(kq, i-g)$ và tìm kq tiếp tục ở đoạn bên phải $T[g]$

- Nếu $T[i]-T[g]<K$ thì tìm kq ở đoạn bên trái $T[g]$

Code viết bằng ngôn ngữ lập trình C++

```
#include <bits/stdc++.h>
#define N int(1e5)
using namespace std;
int a[N+5], s[N+5]; int n, k, kq = N+1;
int timvitri(int k, int d, int c, int i)
{
    int l = -1;
    while (d <= c)
    {
        int g = (d+c)/2;
        if (s[i] - s[g] >= k)
        {
            l = g;
            d = g + 1;
        }
        else c = g - 1;
    }
    return l;
}
int main()
{
    freopen("sub.inp", "r", stdin);
    freopen("sub.out", "w", stdout);
    cin >> n >> k;
    for (int i=1; i<=n; i++)
    {
        cin >> a[i];
        s[i] = s[i-1] + a[i];
    }
    for (int i=1; i<=n; i++){
        int p = timvitri(k, 1, i, i);
        if (p >= 0) kq = min(kq, i-p);
    }
    cout << kq;
    return 0;
}
```

Code viết bằng ngôn ngữ lập trình Python

```
def timvitri(k,d,c,i):
    l = -1;
    while (d <= c):
        g = (d+c)//2;
        if (res[i] - res[g] >= k):
            l = g;
            d = g + 1;
        else: c = g - 1
    return l;
fi=open("sub.inp","r")
n,k=map(int,fi.readline().split())
a=list(map(int,fi.readline().split()))
res = [sum(a[: i + 1]) for i in range(len(a))]
fi.close()
kq=1e9
for i in range(0,len(res)):
    p = timvitri(k,0,i,i);
    #tim vi tri p gan nhat sao cho s[i]-s[p]>=k
    if (p >=0):
        kq=min(kq,i-p);
fo=open("sub.out","w")
print(kq)
fo.close()
```

Hướng giải quyết 3: Tìm kiếm bằng 2 con trỏ - Two Pointe

Độ phức tạp thuật toán: $O(n)$

Phân tích bài toán: Tìm đoạn con liên tiếp ngắn nhất có tổng $\geq K$;

Ý tưởng:

Gọi $\text{sum}(i,j)$ là tổng các phần tử trong đoạn $[i,j]$, một đoạn con $[i,j]$ ngắn nhất nếu $\text{sum}(i,j) \geq K$;

Qua đây bài toán sẽ là bài toán

Vì a dãy dương cho nên:

$+ \text{sum}(1, j) > \text{sum}(2, j) > \text{sum}(3, j) > \text{sum}(4, j) > \dots > \text{sum}(j-1, j) > \text{sum}(j, j)$.

+ Nếu đoạn con $[i,j]$ là đoạn có $\text{sum}(i,j) \geq K$ thì xét đoạn $[i,j+1]$ cũng có $\text{sum}(i,j+1) \geq K$ ta cần xét đoạn $[i-1,j]$ có thỏa mãn bài toán không, tức là đi tìm đoạn ngắn tốt nhất.

+ Nếu đoạn con $[i,j]$ là đoạn có $\text{sum}(i,j) < K$ thì đoạn $[i,j+1]$ xem $\text{sum}(i,j+1) \geq K$ không tức là ta cần bổ sung vào đoạn đó để $\text{sum} \geq K$.

Giải pháp.

- Hai con trỏ i và j đặt ở vị trí đầu $i=1, j=1$ con trỏ i di chuyển nhanh hơn, con trỏ j di chuyển chậm hơn

- Di chuyển con trỏ i từ đầu đến cuối (1 đến n)

Sau mỗi di chuyển nếu :

+ $\text{sum}(i, j) < K$ thì giữ nguyên con trỏ j

+ $\text{sum}(i, j) \geq K$ thì cập nhật đoạn ngắn nhất và tăng dần con trỏ j cho đến khi $\text{sum}(i,j) < K$.

- Đưa ra độ dài đoạn con ngắn nhất của bài toán

Link mô phỏng thuật toán: <https://youtu.be/zxN2dvZxapQ>

Code viết bằng ngôn ngữ lập trình C++

```
#include<bits/stdc++.h>
using namespace std;
int s,lmin,n,a[1000005];long long t;
int main(int argc, char *argv[])
{
    freopen("sub.inp", "r", stdin);
    freopen("sub.out", "w", stdout);
    cin>>n;cin>>s;
    for (int i=1;i<=n;i++) cin>>a[i];
    lmin=1e9;int j= 1;
    for (int i=1;i<=n;i++){
        t = t + a[i];
        while(t>=s){
            lmin=min(lmin,i-j+1);
            t= t - a[j];j++; }
    }
    cout<<lmin;
    return 0;
}
```

Code viết bằng ngôn ngữ lập trình Python

```
fi=open("sub.inp","r")
n,s=map(int,fi.readline().split())
a=list(map(int,fi.read().split()))
fi.close()
d,c,t=0,1,0
lmin=1e9
while(d<c and c<=n-1):
    if(t<s):
        t=t+a[c]
        c=c+1
    else:
        if(lmin>c-d-1):
            lmin=c-d-1
        d=d+1
        t=t-a[d]
fo=open("sub.out","w")
fo.write(str(lmin))
fo.close()
```

Nhận xét: Qua ba hướng giải quyết trên thì hướng giải quyết 1 có độ phức tạp là $O(n^2)$ hướng giải quyết này không đáp ứng được thời gian, hướng giải quyết 2 độ phức tạp $O(n \log n)$ và hướng giải quyết 3 có độ phức tạp $O(n)$. Cả hai hướng giải

quyết này đều đáp ứng được thời gian tuy nhiên giữa hướng giải quyết 2 và hướng giải quyết 3 thì hướng giải quyết 3 tối ưu nhất.

Ví dụ 3. Trò chơi với dãy số - Seqgame

Hai bạn học sinh trong lúc nhàn rỗi nghĩ ra trò chơi sau đây. Mỗi bạn chọn trước một dãy số gồm N số nguyên. Giả sử dãy số mà bạn thứ nhất chọn là: $b_1, b_2, \dots, b_n$, còn dãy số mà bạn thứ hai chọn là: $c_1, c_2, \dots, c_n$

Mỗi lượt chơi mỗi bạn đưa ra một số hạng trong dãy số của mình. Nếu bạn thứ nhất đưa ra số hạng b_i ($1 \leq i \leq N$), còn bạn thứ hai đưa ra số hạng c_j ($1 \leq j \leq N$) thì giá của lượt chơi đó sẽ là $|b_i + c_j|$.

Ví dụ: Giả sử dãy số bạn thứ nhất chọn là 1, -2; còn dãy số mà bạn thứ hai chọn là 2, 3. Khi đó các khả năng có thể của một lượt chơi là (1,2), (1,3), (-2,2), (-2,3). Như vậy, giá nhỏ nhất của một lượt chơi trong số các lượt chơi có thể là 0 tương ứng với giá của lượt chơi (-2,2).

Yêu cầu

Hãy xác định giá nhỏ nhất của một lượt chơi trong số các lượt chơi có thể.

Dữ liệu vào từ tệp **SEQGAME.INP** gồm:

- Dòng đầu tiên chứa số nguyên dương N ($N \leq 10^5$)
- Dòng thứ hai chứa dãy số nguyên $b_1, b_2, \dots, b_n$ ($|b_i| \leq 10^9, i=1, 2, \dots, N$)
- Dòng thứ ba chứa dãy số nguyên $c_1, c_2, \dots, c_n$ ($|c_i| \leq 10^9, i=1, 2, \dots, N$)

Hai số liên tiếp trong một dòng được ghi cách nhau bởi dấu cách.

Kết quả ghi vào tệp **SEQGAME.OUT** là:

- Giá nhỏ nhất tìm được.

Ràng buộc

- 60% số test ứng với 60% số điểm của bài có $1 \leq N \leq 1000$

Ví dụ

SEQGAME.INP	SEQGAME.OUT
2	0
1 -2	
2 3	

Hướng giải quyết 1: Dùng 2 for lồng nhau.

Độ phức tạp của thuật toán: $O(n^2)$

Duyệt tất cả các trường hợp và cập nhật giá trị nhỏ nhất

Code viết bằng ngôn ngữ lập trình C++

```
#include<bits/stdc++.h>
using namespace std;
int b[100005], c[100005], n, lmin=2e9;
int main() {
    cin>>n;
    for (int i=1; i<=n; i++) cin>>b[i];
    for (int i=1; i<=n; i++) cin>>c[i];
    for (int i=1; i<=n; i++)
        for (int j=1; j<=n; j++) {
            lmin=min(lmin, abs(b[i]+c[j]));
        }
    cout<<lmin<<endl;
}
```

Code viết bằng ngôn ngữ lập trình Python

```
n=int(input())
b=list(map(int,input().split()))
c=list(map(int,input().split()))
lmin=1e9
for i in range(0,n):
    for j in range(0,n):
        lmin=min(lmin,abs(b[j]+c[i]))
print(lmin)
```

Hướng giải quyết 2: Tìm kiếm nhị phân - Binary Search

Độ phức tạp của thuật toán: $O(n \log n)$

Đầu tiên ta sẽ sắp xếp dãy a tăng dần. Sau đó, duyệt lần lượt các phần tử b, với mỗi giá trị b[i], ta sử dụng chèn nhị phân trên dãy a để tìm phần tử bé nhất mà lớn hơn hoặc bằng $-b[i]$ và phần tử lớn nhất bé hơn hoặc bằng b[i]. Sau đó chỉ lấy min giá trị tuyệt đối của các tổng số tìm được với b[i].

Code viết bằng ngôn ngữ lập trình C++

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e5 + 1;
int n, a[N], b[N], ans;
int main() {
    freopen("SEQGAME.inp", "r", stdin);
    freopen("SEQGAME.out", "w", stdout);
    cin >> n;
    for(int i=1; i<=n; i++) cin>>a[i];
    for(int i=1; i<=n; i++) cin>>b[i];
    sort(a + 1, a + n + 1);
    long long ans = 2e9;
    for (int i=1; i<=n; i++) {
        int v = -b[i];
        int L = 1, R = n;
        int tmp = a[1];
        while(L <= R) {
```

Code viết bằng ngôn ngữ lập trình Python

```
fi=open("SEQGAME.inp","r")
n=int(fi.readline())
a=list(map(int,fi.readline().split()))
b=list(map(int,fi.readline().split()))
a.sort();
ans=2e9
tmp=0
fi.close()
fo=open("SEQGAME.out","w")
for i in range(0,n-1):
    v = -b[i];
    L,R =0,n-1
    tmp = a[1]
    while(L <= R):
```

<pre> int mid = (L + R)/2; if(a[mid] >= v) { R = mid - 1; tmp = a[mid]; }else L = mid + 1; } ans=min(ans,1LL *abs(tmp+b[i])); L = 1 , R = n; while(L <= R) { int mid = (L + R)/2; if(a[mid] <= v) { L = mid + 1; tmp = a[mid]; }else R = mid - 1; } ans=min(ans,1LL *abs(tmp+b[i])); } cout << ans; return 0; } </pre>	<pre> mid = (L + R) >> 1 if(a[mid] >= v): R = mid - 1; tmp = a[mid] else: L = mid + 1; ans = min(ans,abs(tmp + b[i])); L,R =0, n-1; while(L <= R): mid = (L + R) >> 1 if(a[mid] <= v): L = mid + 1; tmp = a[mid] else: R = mid - 1; ans = min(ans,abs(tmp + b[i])); fo.write(str(ans)) fo.close() </pre>
---	---

Hướng giải quyết 3: Dùng thuật toán 2 con trỏ - Two Pointe

Độ phức tạp của thuật toán: $O(n \log n)$

Ý tưởng: Sắp xếp hai dãy tăng dần thì ta có:

$$b_1 < b_2 < b_3 < b_4 < \dots < b_n$$

$$c_1 < c_2 < c_3 < c_4 < \dots < c_n$$

Để $|b_i + c_j|$ là nhỏ nhất thì $b_i + c_j$ tiến về gần giá trị 0. Do sắp xếp hai dãy là dãy tăng dần.

Xét - Nếu $b_i + c_j \geq 0$ thì muốn tiến gần về 0 thì giảm j

- Nếu $b_i + c_j < 0$ thì muốn tiến gần về 0 cần tăng giá trị i .

Thực hiện như vậy cho đến khi cập nhật $|b_i + c_j|$ nhỏ nhất.

Giải pháp:

+ Sắp xếp tăng dần từng dãy số

+ Dùng 2 biến chạy xuôi và ngược là i và j . Biến i dùng duyệt xuôi mảng $b(N)$ bắt đầu từ vị trí 1, biến j dùng duyệt ngược mảng $c(N)$ bắt đầu từ vị trí N .

+ Vòng lặp thực hiện trong khi $(i \leq N)$ và $(j \geq 1)$:

- Tính $v = b_i + c_j$

- Nếu $v \geq 0$ thì giảm j (để $|v|$ giảm đi)

Ngược lại nếu $v < 0$ thì tăng i (vì số âm khi được tăng lên thì trị tuyệt đối của nó giảm đi).

Link mô phỏng thuật toán: <https://youtu.be/kVt7CK7PabA>

Code viết bằng ngôn ngữ lập trình C++

```
#include<bits/stdc++.h>
using namespace std;
const int N=1e5+2;
int n,b[N],c[N],res,j,i;
int main(){
    cin>>n;
    for (int i=1;i<=n;i++) cin>>b[i];
    for (int i=1;i<=n;i++) cin>>c[i];
    sort(b+1,b+1+n);sort(c+1,c+1+n);
    res=2e9; int i=1,j=n;
    while (i<=n&& j>=1){
        if(res>abs(b[i]+c[j]))res=abs(b[i]+c[j]);
        if (res==0) break;
        if (b[i]+c[j]>0) j--;
        else i++;
    }
    cout<<res;
    return 0;
}
```

Code viết bằng ngôn ngữ lập trình Python

```
fi=open("SEQGAME.inp","r")
n=int(fi.readline())
a=list(map(int,fi.readline().split()))
b=list(map(int,fi.readline().split()))
i,j=0,n-1
res=1e9
a.sort()
b.sort()
while(i<n and j>=0):
    if(res>abs(a[i]+b[j])):
        res=abs(a[i]+b[j])
    if(res==0):
        break
    if(a[i]+b[j]>0):
        j=j-1
    else:
        i=i+1
fi=open("SEQGAME.out","w")
fi.write(str(res))
fi.close()
```

Nhận xét: Qua ba hướng giải quyết trên thì hướng giải quyết 1 có độ phức tạp là $O(n^2)$ hướng giải quyết này không đáp ứng được thời gian, hướng giải quyết 2 độ phức tạp $O(n \log n)$, hướng giải quyết 3 khi sử dụng 2 con trỏ có độ phức tạp $O(n)$, nhưng do chúng ta đã sắp xếp nên mất $O(n \log n)$ thời gian vậy độ phức tạp là $O(n \log n)$. Cả hai hướng giải quyết này đều đáp ứng được thời gian tuy nhiên hướng giải quyết 3 ngắn gọn và dễ hiểu hơn.

2.3. Rèn luyện kỹ năng vận dụng thuật toán 2 con trỏ để giải một số bài toán cơ bản đến nâng cao

2.3.1. Một số bài tập về 2 con trỏ, một con trỏ ở đầu và một con trỏ ở cuối di chuyển vào giữa cho đến khi cả 2 gặp.

Bài 1: Bình phương dãy số.

Cho một mảng số nguyên gồm N phần tử được sắp xếp theo thứ tự không giảm, trả về một mảng bình phương của mỗi số được sắp xếp theo thứ tự không giảm.

Dữ liệu vào từ tệp **BinhPhuong.inp** gồm:

- Dòng thứ 1: Gồm số nguyên N
- Dòng thứ 2: Gồm dãy số nguyên $a_1, a_2, a_3, \dots, a_N$

Dữ liệu ra: Ghi kết quả theo yêu cầu của bài vào tệp **BinhPhuong.out**

Ví dụ

BinhPhuong.inp	BinhPhuong.out
5	0 1 9 16 100
-4 -1 0 3 10	

Hạn chế:

- $1 \leq N \leq 10^4$
- $-10^4 \leq a[i] \leq 10^4$

Hướng giải quyết: Sử dụng thuật toán hai con trỏ

Độ phức tạp thuật toán: $O(n)$

Ý tưởng:

Do dãy đã cho sắp xếp tăng dần sử dụng thuật toán 2 con trỏ, một con trỏ i đầu mảng và con trỏ j cuối mảng. Trong khi i chưa gặp j thì thực hiện như sau:

- Nếu bình phương phần tử cuối mảng $a[j]*a[j]$ lớn hơn bình phương phần tử đầu $a[i]*a[i]$ thì lưu $a[j]*a[j]$ vào mảng khác và giảm $j--$

- Nếu bình phương phần tử đầu mảng $a[i]*a[i]$ lớn hơn bình phương phần tử đầu $a[j]*a[j]$ thì lưu $a[i]*a[i]$ vào mảng khác và tăng $i++$

Code viết bằng ngôn ngữ lập trình C++

```
#include <bits/stdc++.h>
using namespace std;
int a[10000]; int n, b[10000];
int main(int argc, char *argv[])
{
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> a[i];
    int i = 1, j = n, r = n;
    while (i <= j) {
        int left = a[i] * a[i];
        int right = a[j] * a[j];
        if (right > left) {
            b[r] = right;
            j--; r--;
        }
        else {
            b[r] = left; i++; r--;
        }
    }
    for (int i = 1; i <= n; i++)
        cout << b[i] << " ";
}
```

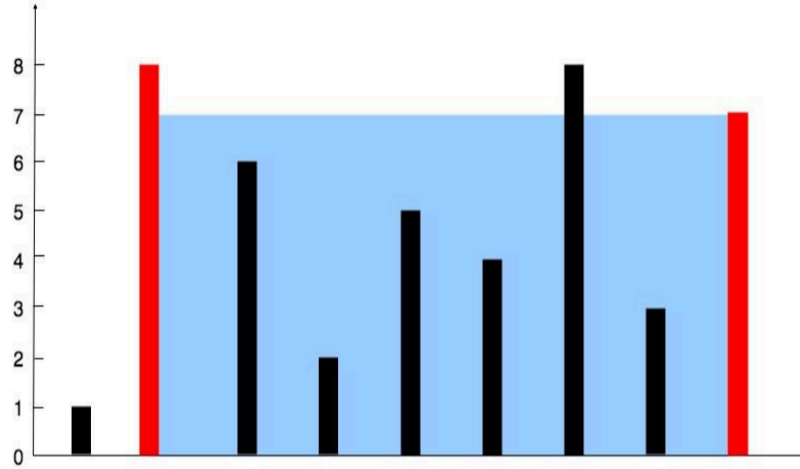
Code viết bằng ngôn ngữ lập trình Python

```
n = int(input())
b = []
a = list(map(int, input().split()))
i, j, k = 0, len(a) - 1, len(a) - 1
while (i <= j):
    l = a[i] * a[i]
    r = a[j] * a[j]
    if (r > l):
        b.insert(k, r)
        j = j - 1
        k = k - 1
    else:
        b.insert(k, l)
        i = i + 1
        k = k - 1
b.reverse()
print(b)
```

BÀI 2. Thùng chứa nhiều nước nhất

Cho bạn n số nguyên không âm $a_1, a_2, a_3, \dots, a_n$, mỗi số đại diện cho một điểm (i, a_i) trong tọa độ. Vẽ n đường thẳng đứng theo tọa độ hai điểm cuối của đường thẳng đứng thứ i là (i, a_i) và $(i, 0)$. Tìm hai trong số các đoạn thẳng để cái thùng mà chúng tạo thành với trục x có thể chứa được nhiều nước nhất.

Giải thích: Bạn không thể nghiêng thùng chứa và giá trị của n ít nhất là 2.



Đường thẳng đứng trong hình đại diện cho mảng đầu vào $[1, 8, 6, 2, 5, 4, 8, 3, 7]$. Trong trường hợp này, giá trị lớn nhất mà thùng chứa có thể chứa nước (được hiển thị dưới dạng phần màu xanh lam) là 49.

Ví dụ:

Đầu vào: $[1, 8, 6, 2, 5, 4, 8, 3, 7]$

Đầu ra: 49

Hướng giải quyết : Sử dụng thuật toán hai con trỏ

Độ phức tạp về thời gian: $O(n)$

Từ giải pháp brute force, chúng ta có thể thấy rằng chúng ta đang sử dụng hai vòng lặp với hai chỉ số để lặp lại tất cả các phần tử. Các chỉ số này sẽ duyệt tất cả các trường hợp nhưng nó có thể gây dư thừa.

Giải pháp:

- Giữ hai con trỏ $i = 0$ và con trỏ $j = n-1$ và một giá trị maxW lưu trữ diện tích tối đa.
- Trong khi i còn nhỏ hơn j ta thực hiện:
 - + Cập nhật max_area với tối đa max_area và $\min(a[i], a[j]) * (j-i)$

+ Nếu giá trị tại $a[i]$ lớn hơn $a[j]$ thì giảm $j --$ ngược lại nếu $a[i]$ nhỏ hơn $a[j]$ tăng giá trị $i++$

- Đưa ra diện tích tối đa.

Code viết bằng ngôn ngữ lập trình C++

```
#include<bits/stdc++.h>
using namespace std;
int main(int argc, char *argv[])
{
    int a[10000],n; cin>>n;
    int minH=1e5,maxW=-1e5;
    for (int i=0;i<n;i++) cin>>a[i];
    int i = 0,j = n - 1;
    while (i<j){
        maxW=max(maxW,min(a[i],a[j])*(j-i));
        if (a[i] < a[j]) {
            ++i;
        }
        else {
            --j;
        }
    }
    cout<<maxW;
}
```

Code viết bằng ngôn ngữ lập trình Python

```
def maxArea():
    ans = 0
    l = 0
    r = len(a) - 1
    while l < r:
        minHeight = min(a[l],a[r])
        ans=max(ans,minHeight*(r-l))
        if a[l] < a[r]:
            l += 1
        else:
            r -= 1
    return ans
n=int(input())
a=list(map(int,input().split()))
print(maxArea())
```

2.3.2. Một số bài tập về một con trỏ di chuyển chậm và một con trỏ di chuyển với tốc độ nhanh hơn.

Bài 1: Thời gian đọc sách- READBOOK

Nam muốn đi đọc sách vào thời gian được nghỉ buổi trưa ở công ty. Hôm nay anh ấy có thời gian rảnh là s và muốn đọc n cuốn sách. Mỗi cuốn sách sẽ đọc hết trong khoảng thời gian a_i và Nam sẽ đọc lần lượt các cuốn sách theo thứ tự từ trái qua phải. Hãy giúp Nam tìm ra số quyển sách nhiều nhất mà Nam có thể đọc xong trong khoảng thời gian nghỉ trưa này.

Dữ liệu vào từ tệp **READBOOK.INP** gồm:

- Dòng đầu tiên là số n quyển sách có trong nhà sách và thời gian rảnh s . ($1 \leq n \leq 10^5$; $1 \leq s \leq 10^9$).

- Dòng thứ 2 là khoảng thời gian đọc của mỗi quyển sách $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^4$)

Dữ liệu ra ghi vào tệp **READBOOK.OUT** là:

- Số quyển sách nhiều nhất mà Nam có thể đọc trong thời gian t

Ví dụ:

READBOOK.INP	READBOOK.OUT
4 5 3 1 2 1	3

Hướng giải quyết: Sử dụng thuật toán hai con trỏ

Độ phức tạp về thời gian: $O(n)$

Giải pháp:

Bài toán quy về tìm đoạn con liên tiếp dài nhất có tổng $\leq s$. Sử dụng thuật toán 2 con trỏ

- Đặt hai con trỏ i và j sẽ đặt ở đầu mảng
- Di chuyển lần lượt con trỏ j từ 1 đến n .
 - Sau mỗi lần di chuyển con trỏ i , nếu
 - $sum(i, j) \leq s$: giữ nguyên vị trí con trỏ i .
 - $sum(i, j) > s$: tăng dần vị trí con trỏ i cho đến khi $sum(i, j) \leq s$.
 - Hiện tại với vị trí con trỏ i và j , ta biết đoạn tốt dài nhất với vị trí kết thúc tại j là đoạn $[i, j]$.
- Độ dài đoạn con tốt dài nhất chính là giá trị độ dài lớn nhất của các đoạn tốt dài nhất với vị trí kết thúc tại i , với mỗi i từ 1 đến n .

Code viết bằng ngôn ngữ lập trình C++

```
#include<bits/stdc++.h>
using namespace std;
long long n,s;int a[1000005];
int main(int argc,char *argv[])
{
    cin>>n;cin>>s;
    for(int i=0;i<n;i++) cin>>a[i];
    int ans = 0, sum = 0;
    for (int i = 0, j = 0; j<n;j++)
    {
        sum += a[j];
        while (sum > s) {
            sum -= a[i];
            i++;
        }
        ans = max(ans,j - i + 1);
    }
    cout << ans;
}
```

Code viết bằng ngôn ngữ lập trình Python

```
n,s=map(int,input().split())
a=list(map(int,input().split()))
lmax=-1e9
t,i=0,0
for j in range(0,n):
    t=t+a[j]
    if (t<=s):
        lmax=max(lmax,j-i+1)
    else:
        t=t-a[i]
        i+=1
print(lmax)
```


Bài 2. ĐOẠN MAX

Cho chuỗi kí tự S gồm toàn các chữ cái in hoa (A...Z) với độ dài không vượt quá 10⁴.

Yêu cầu: Hãy tìm đoạn con các kí tự liên tiếp dài nhất sao cho không có kí tự nào xuất hiện nhiều hơn một lần. Trong trường hợp có nhiều hơn một đoạn con có cùng chiều dài dài nhất, hãy chỉ ra đoạn xuất hiện đầu tiên trong chuỗi S.

Dữ liệu vào: từ tệp **DOANMAX.INP**:

- Gồm một dòng duy nhất chứa chuỗi S.

Kết quả: Ghi ra tệp **DOANMAX.OUT**

- Chỉ một dòng duy nhất chứa số nguyên P và L tương ứng là vị trí và chiều dài của đoạn con dài nhất tìm được.

Ví dụ:

DOANMAX.INP	DOANMAX.OUT
ABABCDAC	3 4

Lưu ý: Có 80% số test có độ dài chuỗi S không vượt quá 255.

Giải thích test ví dụ: Đoạn con dài nhất tìm được là ABCD có vị trí 3 và độ dài 4.

Hướng giải quyết: Sử dụng thuật toán hai con trỏ.

Độ phức tạp về thời gian: $O(n \log n)$

Ý tưởng: Bài này có nhiều cách giải nhưng dưới đây chúng tôi giới thiệu thuật toán hai con trỏ kết hợp với cấu trúc dữ liệu kiểu set hoặc map trong C++, tương ứng trong Python thì set và dictionary.

- Duyệt từ trái sang phải bằng 2 con trỏ i và j, con trỏ j di chuyển sang phải trước. Cập nhật độ dài dài nhất của chuỗi con bằng sử dụng biến maxlength và sử dụng set để kiểm tra kí tự đã truy cập

- Nếu kí tự $s[j]$ không có trong set thì chèn nó vào set và trượt cửa sổ hiện tại sang phải bằng cách di chuyển con trỏ j, chúng tôi cập nhật độ dài dài nhất của chuỗi con với các kí tự duy nhất là $maxlength = \max(maxlength, j-i)$

- Nếu kí tự $s[j]$ đã có trong set, chúng ta xóa kí tự đó ra khỏi set và trượt cửa sổ sang phải bằng cách di chuyển con trỏ i.

Quá trình sẽ kết thúc khi con trỏ nào đến cuối chuỗi tức là $i=n$ hoặc $j=n$.

Code viết bằng ngôn ngữ lập trình C++

```
#include<bits/stdc++.h>
using namespace std;
string s;int x;
int longstring(string str,int n){
    int maxlength=0;
    int i=0,j=0;
    set<char> h;
    while(i<n&&j<n){
        if(h.count(str[j])==0){
            h.insert(str[j]);
            j++;
            if(maxlength<j-i){
                maxlength=j-i;x=j;}
        }
        else {
            h.erase(str[i]);
            i++;
        }
    }
    return maxlength;
}
int main(){
    freopen("Doanmax.inp","r",stdin);
    freopen("Doanmax.out","w",stdout);
    getline(cin,s);int smax;
    int n=s.size();
    smax=longstring(s,n);
    cout<<x-smax+1<<" "<<smax;
}
```

Code viết bằng ngôn ngữ lập trình Python

```
def longstring(str,n):
    maxlength=0;
    i=0
    j=0
    h=set()
    while(i<n and j<n):
        if(str[j] not in h):
            h.add(str[j])
            j+=1
            if(maxlength<j-i):
                maxlength=j-i
                x=j
        else:
            h.remove(str[i])
            i+=1
    return maxlength,x
fi=open("Doanmax.inp","r")
s=fi.readline()
fi.close()
n=len(s)
fo=open("Doanmax.out","w")
smax,x=longstring(s,n);
fo.write(str(x-smax+1)+' '+str(smax))
fo.close()
```

Bài 3: Đèn màu

Thành phố Hải Dương đang chuẩn bị kỷ niệm ngày giải phóng (30/10) nên các con đường lung linh dây đèn màu. Dọc con đường từ rạp chiếu phim về nhà, Dũng và Mai thấy một dãy n chùm đèn màu cách đều nhau đánh số lần lượt 1, 2, ..., n . Mỗi chùm đèn có một màu sắc riêng và dễ đơn giản, có thể coi nó như là một số nguyên dương. Có tất cả m giá trị màu khác nhau. Dũng muốn chụp một bức ảnh gồm các chùm đèn liên tiếp của dãy chùm đèn này. Trong bức ảnh m giá trị màu của các chùm đèn đều xuất hiện. Để chất lượng ảnh tốt, Dũng muốn số lượng chùm đèn màu có trong ảnh là **nhỏ nhất có thể**. Mai đã giúp Dũng tìm ra được dãy chùm đèn màu thỏa mãn các yêu cầu trên bằng cách quan sát trực tiếp.

Còn bạn? Chỉ cần biết dãy số mô tả màu của dãy chùm đèn màu bạn có thể cho biết số lượng chùm đèn màu có trong bức ảnh của Dũng được không?

Yêu cầu: Cho biết các giá trị n, m và dãy màu $a_1, a_2, \dots, a_n$. Hãy xác định số chùm đèn màu có trong bức ảnh của Dũng

Dữ liệu vào: từ tệp **DENMAU.INP** gồm:

- Dòng 1: Chứa 2 số nguyên dương n, m ($n \leq 10^5, m \leq n$)
- Dòng 2: Chứa m số nguyên dương $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^6$)

Hai số liên tiếp trên cùng một dòng của file **DENMAU.INP** cách nhau một dấu cách trống

Kết quả: Ghi ra tệp **DENMAU.OUT**

- Một số nguyên là số lượng chùm đèn màu có trong bức ảnh của Dũng.

Ví dụ:

DENMAU.INP	DENMAU.OUT
7 3 1 2 2 1 1 3 3	4

Giải thích: Dũng chụp bức ảnh chứa các chùm đèn màu 3,4,5,6

Ghi chú: Bài được chấm qua 6 test, mỗi test 0,25 điểm. Trong đó:

- 1 test giá trị màu không vượt quá 2
- 1 test giá trị màu không vượt quá 3
- 2 test có $n \leq 1000$
- 2 test không có ràng buộc bổ sung

Hướng giải quyết: Sử dụng thuật toán hai con trỏ.

Độ phức tạp thuật toán: $O(n)$

Ý tưởng: Sử dụng thuật toán hai con trỏ.

Do bài toán yêu cầu $1 \leq a_i \leq 10^6$ nên ta sử dụng mảng đếm duy trì xuất hiện phần tử, nếu $a_i \leq 10^9$ trong C++ ta dùng map (còn Python dùng dictionary)

Xét đoạn $a[i], a[i+1], a[i+2], a[i+3], \dots, a[j]$

- Nếu đoạn này chưa đủ m giá trị khác nhau thì ta bổ sung thêm vào đoạn để hy vọng có đủ m giá trị khác nhau. Việc bổ sung này sẽ thực hiện cuối đoạn

- Nếu đoạn này đã đủ m giá trị khác nhau thì ta làm 2 việc sau:

- Cập nhật đoạn ngắn nhất.
- Hy vọng có thể tìm được đoạn ngắn hơn bằng cách loại bỏ giá trị ở đầu dãy.

Code viết bằng ngôn ngữ lập trình C++

```

#include<bits/stdc++.h>
#define amax 1000001
#define N 100002
using namespace std;
int n,m,cm,res; int c[amax],a[N];
int main(int argc, char *argv[])
{
    ios_base::sync_with_stdio(false);
    cin.tie(0);cout.tie(0);
    freopen("bai4.inp","r",stdin);
    freopen("bai4.out","w",stdout);
    cin>>n;cin>>m;
    for (int i=1;i<=n;i++) cin>>a[i];
    int i=1,j=1,cm=1,res=n;
    c[a[1]]=1;
    while (i<=j&&j<=n)
        if(cm<m)
        {
            j++;
            if(c[a[j]]==0) cm++;c[a[j]]++;
        }
        else{
            res=min(res,j-i+1);
            c[a[i]]--;
            if (c[a[i]]==0) cm--;
            i++;
        }
    cout<<res;
    return 0;
}

```

Code viết bằng ngôn ngữ lập trình Python

```

fi=open("bai4.inp","r")
n,m=map(int,fi.readline().split())
a=list(map(int,fi.readline().split()))
fi.close()
a=[0]+a+[0]
c=[0]*1000001
res=n
i,j,cm=1,1,1
c[a[1]]=1
while (i<=j and j<=n):
    if(cm<m):
        j+=1
        if (c[a[j]]==0):cm+=1
        c[a[j]]+=1
    else:
        res=min(res,j-i+1)
        c[a[i]]-=1
        if (c[a[i]]==0):cm-=1
        i+=1
fo=open("bai4.out","w")
fo.write(str(res))
fo.close()

```

2.3.3. Hai con trỏ di chuyển trên hai mảng hoặc xâu

Bài 1. Cho hai dãy số nguyên được sắp xếp theo thứ tự không giảm, dãy thứ nhất gồm N số nguyên $a_1, a_2, \dots, a_N$ và dãy thứ hai gồm M số nguyên $b_1, b_2, \dots, b_M$. Đối với mỗi phần tử của dãy. Tìm số phần tử thỏa mãn yêu cầu số lượng phần tử dãy thứ nhất ít hơn từng phần tử dãy thứ 2.

Dữ liệu vào: từ tệp **Bai1.inp** gồm 3 dòng

- Dòng đầu tiên chứa các số nguyên N và M, kích thước của mảng ($1 \leq N, M \leq 10^5$).
- Dòng thứ hai chứa N số nguyên $a_1, a_2, \dots, a_N$ ($-10^9 \leq a_i \leq 10^9$)
- Dòng thứ ba chứa M số nguyên $b_1, b_2, \dots, b_M$ ($-10^9 \leq b_j \leq 10^9$)

Dữ liệu ra: Ghi kết quả vào tệp **Bai1.out**

- In ra M số, số phần tử của mảng đầu tiên ít hơn mỗi phần tử của mảng thứ hai.

Ví dụ

Bai1.inp	Bai1.out
6 7	1 1 2 3 4 6 6
1 6 9 13 18 18	
2 3 8 13 15 21 25	

Hướng giải quyết: Sử dụng thuật toán hai con trỏ.

Độ phức tạp thuật toán: $O(n)$

Giải pháp:

- Sử dụng 2 con trỏ $i=0$ ở đầu mảng thứ nhất, $j=0$ ở đầu mảng thứ 2 di chuyển 2 con trỏ về phía sau trong khi i, j chưa đến cuối mảng thực hiện như sau:

+ So sánh nếu a_i nhỏ hơn b_j tăng i một đơn vị ngược lại tăng j một đơn vị.

- Trong khi j nhỏ hơn m tăng j lên và đưa kết quả ra (tức là số lượng phần tử còn lại trong mảng thứ 2 luôn lớn mảng thứ nhất).

Code viết bằng ngôn ngữ lập trình C++

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m; cin >> n >> m;
    int a[n], b[m];
    for(int &x : a) cin >> x;
    for(int &x : b) cin >> x;
    int i = 0, j = 0, cnt = 0;
    while(i < n && j < m) {
        if(a[i] < b[j]) {
            ++i;
        }
        else {
            cout << i << " ";
            ++j;
        }
    }
    while(j < m) {
        cout << n << " ";
    }
}
```

Code viết bằng ngôn ngữ lập trình Python

```
n, m = map(int, input().split())
a = list(map(int, input().split()))
b = list(map(int, input().split()))
i, j = 0, 0
while (i < n and j < m):
    if(a[i] < b[j]):
        i = i + 1
    else:
        print(i, end=" ")
        j = j + 1
while(j < m):
    print(n, end=" ")
    j = j + 1
```

Bài 2. Cho hai dãy số nguyên A và dãy B, dãy A gồm N số nguyên $a_1, a_2, \dots, a_N$ và dãy B gồm M số nguyên $b_1, b_2, \dots, b_M$. Yêu cầu bài toán tìm một phần tử trong mảng A là $A[i]$ và một phần tử khác trong mảng B là $B[j]$, sao cho hiệu số giữa $A[i]$ và $B[j]$ ($|A[i] - B[j]|$) càng nhỏ càng tốt, trả về hiệu số nhỏ nhất của chúng.

Dữ liệu vào: Từ tệp **Bai2.inp** gồm 3 dòng:

- Dòng đầu tiên chứa các số nguyên N và M, kích thước của mảng ($1 \leq N, M \leq 10^5$).
- Dòng thứ hai chứa N số nguyên $a_1, a_2, \dots, a_N$ ($-10^9 \leq a_i \leq 10^9$)
- Dòng thứ ba chứa M số nguyên $b_1, b_2, \dots, b_M$ ($-10^9 \leq b_j \leq 10^9$)

Dữ liệu ra: Lưu vào tệp **Bai2.out**

- In ra hiệu nhỏ nhất.

Ví dụ :

Bai2.inp	Bai2.out	Giải thích
4 4 3 6 7 4 2 8 9 3	0	$A[0] - B[3] = 0$
4 3 1 2 3 4 7 6 5	1	$B[2] - A[3] = 1$

Hướng giải quyết: Sử dụng thuật toán 2 con trỏ

Độ phức tạp thuật toán: $O(n \log n)$

Giải pháp:

- Sắp xếp hai mảng A và B tăng dần
- Dùng con trỏ i, j đặt đầu 2 mảng, di chuyển 2 con trỏ từ trước ra sau.
- Nếu $A[i] > B[j]$, Nếu muốn hiệu $A[i] - B[j]$ nhỏ, cần di chuyển con trỏ j (tức là tăng j một đơn vị)
- Nếu $A[i] < B[j]$, Nếu muốn hiệu $B[j] - A[i]$ nhỏ, cần di chuyển con trỏ i (tức là tăng i một đơn vị)
- Nếu $A[i] = B[j]$ hiệu bằng 0 và kết thúc, cập nhật giá trị nhỏ nhất.

Code viết bằng ngôn ngữ lập trình C++

```
#include <bits/stdc++.h>
using namespace std;
int hieunhonhat(vector<int>&A,vector<int>&B)
{
    sort(A.begin(), A.end());
    sort(B.begin(), B.end());
    int i = 0, j = 0, min = INT_MAX;
    while(i < A.size() && j < B.size())
    {
        int diff;
        if (A[i] > B[j])
        {
            diff = A[i] - B[j];
            ++j;
        }

        else if (A[i] < B[j])
        {
            diff = B[j] - A[i];
            ++i;
        }
        else return 0;
        if (diff < min) min = diff;
    }
    return min;
}

int main(){
    vector<int> A,B;int x,y,n,m;cin>>n>>m;
    for (int i=0;i<n;i++){
        cin>>x;A.push_back(x);
    }
    for (int i=0;i<m;i++){
        cin>>y;B.push_back(y);
    }
    int lmin=hieunhonhat(A,B);
    cout<<lmin;
}
```

Code viết bằng ngôn ngữ lập trình Python

```
def hieunhonhat(A, B):
    lmina = 1e9
    p1, p2 = 0, 0
    while p1 < len(A) and p2 < len(B):
        if A[p1] > B[p2]:
            diff = A[p1] - B[p2];
            p2 += 1
        else:
            if(A[p1] < B[p2]):
                diff = B[p2] - A[p1];
                p1 += 1
            else:
                return 0

        if lmina > diff:
            lmina = diff;
    return diff

n,m=map(int,input().split())
A=list(map(int,input().split()))
B=list(map(int,input().split()))
A.sort()
B.sort()
lmin=hieunhonhat(A,B)
print(lmin)
```

Bài 3:

Cho hai dãy số nguyên A và dãy B, dãy A gồm N số nguyên $a_1, a_2, \dots, a_N$ và dãy B gồm M số nguyên $b_1, b_2, \dots, b_M$. Yêu cầu tìm một mảng giao của hai mảng A và B, mỗi phần tử trong kết quả phải xuất hiện nhiều lần như nó hiển thị trong cả hai mảng và bạn có thể trả về kết quả theo bất kỳ thứ tự nào.

Dữ liệu vào: Từ tệp **Bai3.inp** gồm 3 dòng:

- Dòng đầu tiên chứa các số nguyên N và M, kích thước của mảng ($1 \leq N, M \leq 10^5$).
- Dòng thứ hai chứa N số nguyên $a_1, a_2, \dots, a_N$ ($-10^9 \leq a_i \leq 10^9$)
- Dòng thứ ba chứa M số nguyên $b_1, b_2, \dots, b_M$ ($-10^9 \leq b_i \leq 10^9$)

Dữ liệu ra: Lưu vào tệp **Bai3.out**

- In ra hiệu nhỏ nhất.

Ví dụ:

Bai3.inp	Bai3.out
4 2	
1 2 2 1	2 2
2 2	

Hướng giải quyết: Sử dụng thuật toán hai con trỏ

Độ phức tạp thuật toán: $O(m \log m + n \log n)$

Giải pháp:

Nếu các phần tử của cả mảng được sắp xếp thì cách tiếp cận này hiệu quả hơn. Đối với vấn đề này vì nó không được sắp xếp.

- Sắp xếp cả hai mảng tăng dần.
- Sử dụng hai con trỏ i và j cho hai mảng và khởi tạo cả hai bằng 0.
- Di chuyển con trỏ i dọc theo mảng thứ nhất (A) và con trỏ j dọc theo mảng thứ hai (B)
- So sánh hai phần tử được trỏ bởi i và j.
- Nếu $A[i]$ nhỏ hơn $B[j]$ thì chúng ta biết rằng chúng ta phải rời khỏi phần tử nhỏ hơn và chuyển đến phần tử tiếp theo (lớn hơn) trong mảng A để kiểm tra giao điểm của các phần tử.
- Ngược lại nếu $A[i]$ lớn hơn $B[j]$ thì tương tự, chúng ta phải chuyển đến phần tử tiếp theo (lớn hơn) trong mảng B.

- Nếu không, cả hai phần tử giao nhau, do đó thêm phần tử này vào mảng kết quả. Và tăng cả i và j.

Code viết bằng ngôn ngữ lập trình C++

```
#include <bits/stdc++.h>
using namespace std;
int n,m,nums1[10000],nums2[10000];
int main(){
    cin>>n>>m; vector <int> a;
    for(int i=0;i<n;i++)cin>>nums1[i];
    for(int i=0;i<m;i++)cin>>nums2[i];
    sort(nums1,nums1+n);
    sort(nums2,nums2+m);
    int i = 0;int j = 0;
    while (i <n&& j <m){
        if (nums1[i]<nums2[j]){ ++i;}
        else if(nums1[i]>nums2[j]){
            ++j;}
        else{
            a.push_back(nums1[i]);
            ++i; ++j;}
    }
    for(i=0;i<a.size();i++)cout<<a[i]<<" ";
}
```

Code viết bằng ngôn ngữ lập trình Python

```
n,m=map(int,input().split())
c=[]
nums1=list(map(int,input().split()))
nums2=list(map(int,input().split()))
nums1.sort()
nums2.sort()
i,j= 0,0
while (i <n and j <m):
    if (nums1[i] < nums2[j]):i+=1;
    else:
        if (nums1[i]>nums2[j]):j+=1
        else:
            c.append(nums1[i])
            i+=1;
            j+=1
print(c)
```

2.4. Bài tập tự giải có hướng dẫn

Dạng 1: Hai con trỏ, một con trỏ ở đầu và một con trỏ ở cuối di chuyển vào giữa cho đến khi cả 2 gặp nhau.

Bài 1: Dãy chẵn lẻ

Cho một mảng A gồm N số nguyên không âm. Yêu cầu in ra một mảng bao gồm tất cả các phần tử chẵn của A đưa ra phía trước theo sau là tất cả các phần tử lẻ của A.

Dữ liệu vào: Tập **Daychanle.inp** gồm:

- Dòng đầu chứa số nguyên N($N \leq 10^4$)
- Dòng thứ 2 chứa dãy số nguyên gồm $a_1, a_2, a_3, \dots, a_N$ ($N \leq 10^4, 0 \leq a_i \leq 10^4$)

Dữ liệu ra: ghi kết quả vào tập **Daychanle.out** là

- Bạn có thể trả về bất kỳ mảng câu trả lời nào thỏa mãn điều kiện này.

Ví dụ:

Daychanle.inp	Daychanle.out	Giải thích
4	2 4 3 1	
3 1 2 4	4 2 3 1	
	2 4 1 3	
	4 2 1 3	

Ý tưởng:

Yêu cầu của bài toán này là xếp số chẵn đưa ra phía trước và số lẻ ra phía sau của một mảng. Chúng ta có thể thấy rằng nó giống như ý tưởng của thuật toán hai con trỏ.

- Dùng con trỏ i đầu mảng, con trỏ j cuối mảng di chuyển 2 con trỏ vào giữa mảng. Khi phần tử phía đầu mảng $a[i]$ là số lẻ còn phần tử phía cuối mảng $a[j]$ số chẵn thì ta đổi chỗ cho nhau, nếu $a[i]$ chẵn tăng i , $a[j]$ lẻ giảm j đến khi $i=j$ dừng.

Code C++ tham khảo theo link: <https://ideone.com/lcq9gv>

Code Python tham khảo theo link: <https://ideone.com/yEYSTp>

Bài 2: Trò chơi bốc bài

An và Bình chơi một trò chơi. Các quy tắc của trò chơi rất đơn giản. Người chơi có n quân bài được đặt liên tiếp thành một hàng ngang trên mặt bàn. Mỗi quân bài chứa một số, tất cả các số trên quân bài là khác biệt. Hai người chơi thay phiên nhau, An đi chuyển trước. Trong lượt của mình, người chơi có thể lấy một quân bài: quân ngoài cùng bên trái hoặc quân ngoài cùng bên phải. Trò chơi kết thúc khi không còn quân bài nữa. Người chơi có tổng số tối đa trên các thẻ của mình vào cuối trò chơi sẽ thắng.

An và Bình, mỗi người trong số họ sẽ chọn thẻ có số lớn hơn trong quá trình thực hiện bốc bài của mình.

Hãy xác định tổng số giá trị của các quân bài của mỗi người khi trò chơi kết thúc.

Dữ liệu vào: từ tệp **CGAME.INP** gồm

- Dòng đầu tiên chứa số nguyên dương n ($n \leq 10^5$)
- Dòng thứ hai chứa n số nguyên dương thể hiện các giá trị của mỗi quân bài.

Dữ liệu ra: lưu kết quả vào tệp **CGAME.OUT** là

- Một dòng duy nhất chứa hai số nguyên là tổng giá trị các quân bài của mỗi người khi trò chơi kết thúc.

Ví dụ:

CGAME.INP	CGAME.OUT
4	12 5
4 1 2 10	

Ý tưởng: Để tính tổng giá trị quân bài của mỗi người bốc thì mỗi lượt bốc người chơi bốc quân bài bên trái hoặc quân bên phải vì vậy chọn quân bài lớn hơn bốc trước, 2 người thay nhau bốc cho đến hết.

Người bốc lượt đầu thay nhau bốc trên đoạn $[1, n]$ gọi $a[i]$ quân bài bên trái, $a[j]$ quân bài bên phải ($1 \leq i < j \leq n$).

- Đặt con trỏ $i=1$ đầu mảng, con trỏ $j=n$ cuối mảng

- Trong khi $i < j$ thì thực hiện như sau:

+ Nếu $a[i] < a[j]$ thì chọn $a[j]$ tức là bổ sung quân bài được chọn vào tổng $s1$ ($s1$ gọi là tổng quân bài được chọn người chơi thứ nhất), đồng thời giảm giá $j--$

+ Ngược lại chọn $a[i]$ bổ sung $a[i]$ vào tổng $s2$ ($s2$ gọi là tổng quân bài được chọn người chơi thứ hai), đồng thời tăng vị trí $i++$.

- Đưa kết quả tổng $s1$ và $s2$ ra.

Code C++ tham khảo theo link: <https://ideone.com/O89zmD>

Code Python tham khảo theo link: <https://ideone.com/ZDqRUb>

Dạng 2: Một số bài tập về một con trỏ di chuyển chậm và một con trỏ di chuyển với tốc độ nhanh hơn.

Bài 1: Thi hoa hậu bò

Bờm có n cô bò, mỗi cô bò thứ i được đặc trưng bởi hai giá trị a_i – nhan sắc và b_i – lượng sữa sản xuất được của cô ấy. Bờm dự định sẽ chọn ra một số cô bò trong n cô để tham gia cuộc thi "hoa hậu bò sữa" với nội dung đồng đội. Tiêu chí đặt ra của Ban tổ chức cuộc thi là ngoài nhan sắc xinh đẹp ra thì tổng lượng sữa sản xuất được của các cô bò cùng một đội phải càng nhiều càng tốt.

Một nhóm các cô bò của Bờm sẽ đồng ý tham gia cuộc thi nếu như các cô bò trong nhóm không cảm thấy xấu hổ. Một cô bò sẽ cảm thấy xấu hổ nếu như nhan sắc của cô ấy kém hơn ít nhất d đơn vị so với những cô bò khác cùng một nhóm.

Bạn hãy giúp Bờm chọn ra một nhóm các cô bò tham gia cuộc thi sao cho không có cô bò nào trong nhóm cảm thấy xấu hổ mà tổng lượng sữa sản xuất được là nhiều nhất.

Dữ liệu vào: từ tệp **COW.INP** gồm:

- Dòng đầu tiên chứa hai số nguyên dương n và d ($1 \leq n \leq 10^5$, $1 \leq d \leq 10^9$)

- n dòng tiếp theo, dòng thứ i chứa hai số nguyên a_i và b_i ($1 \leq a_i, b_i \leq 10^9$)

Dữ liệu ra: ghi kết quả vào tệp **COW.OUT** là:

- Một số nguyên duy nhất là giá trị của tổng lượng sữa tìm được trong nhóm bò của Bờm

Ví dụ:

COW.INP	COW.OUT
8 8	22
4 8	
15 7	
8 2	
13 4	
25 7	
1 4	
16 8	
21 3	

*** Giải thích:**

Bờm sẽ chọn ra các cô bò thứ 2, 4, 7 và thứ 8 để được lượng sữa lớn nhất là:

$$7 + 4 + 8 + 3 = 22$$

Ý tưởng: Dùng cấu trúc dữ liệu pair<int, int> a trong python dùng list

- Sắp xếp cô bò có nhan sắc tăng dần
- Cô bò thứ i có nhan sắc $a[i]$, cô bò thứ j có nhan sắc $a[j]$.
- Nếu đoạn $[i, j]$ là đoạn chứa các cô bò cảm thấy không xấu hổ (tức là $a[j] - a[i] \leq d$) thì đoạn $[i, j-1]$ cũng thỏa mãn. Vậy muốn tìm đoạn có lượng sữa nhiều nhất thì ta bổ sung một cô bò có nhan sắc vào để được lượng sữa nhiều nhất có thể
- Nếu đoạn $[i, j]$ không phải đoạn chứa các cô bò cảm thấy xấu hổ thì cần xét đoạn mới có cảm thấy xấu hổ không (tức bỏ cô bò $a[i]$ ra khỏi xét đoạn $[i+1, j]$).

Code C++ tham khảo theo link: <https://ideone.com/TGEyLI>

Code Python tham khảo theo link: <https://ideone.com/ZwAlde>

Bài 2: Hai giá trị

Cho dãy số nguyên $a_1, a_2, \dots, a_N$. Tìm độ dài đoạn con dài nhất của dãy chỉ bao gồm hai giá trị.

Dữ liệu: Vào từ file văn bản **TWOVALS.INP** gồm:

- Dòng 1: số nguyên N ($1 \leq N \leq 10^6$);
- Dòng 2: N số nguyên $a_1, a_2, \dots, a_N$ ($1 \leq a_i \leq 10^9$).

Kết quả: Ghi ra file văn bản **TWOVALS.OUT** một số nguyên duy nhất là độ dài đoạn con dài nhất chỉ bao gồm hai giá trị theo phương án tìm được.

Các số trên một dòng của input file được ghi cách nhau bởi ít nhất một dấu cách

Ví dụ

TWOVALS.INP	TWOVALS.OUT
7 1 3 2 3 3 1 2	4

Ý tưởng:

- Nếu đoạn $[i, j]$ chứa 2 giá trị được bắt đầu vị trí i và kết thúc vị trí thứ j ta hy vọng đoạn $[i, j+1]$ cũng chứa 2 giá trị tức là xét tiếp một phần tử tiếp đó có thỏa mãn 2 giá trị không để ta được đoạn con dài nhất chứa hai giá trị nếu thỏa mãn thì ta cập nhật đoạn đó.

- Nếu đoạn $[i, j]$ hơn 2 giá trị ta cần loại bỏ 1 giá trị ở đầu đoạn đang xét tức là xét tiếp đoạn $[i+1, j]$ để tìm đoạn con mới.

Code C++ tham khảo theo link: <https://ideone.com/QzzSdR>

Code Python tham khảo theo link: <https://ideone.com/JDj0xX>

Dạng 3: Hai con trở di chuyển trên hai mảng hoặc xâu

Bài 1. Cho hai dãy số nguyên được sắp xếp theo thứ tự không giảm, dãy thứ nhất gồm n số nguyên $a_1, a_2, \dots, a_n$ và dãy thứ hai gồm m số nguyên $b_1, b_2, \dots, b_m$. Tìm số các cặp (i, j) sao cho $a_i = b_j$.

Dữ liệu vào từ tệp Bai1.inp gồm:

- Dòng đầu tiên chứa các số nguyên n và m , kích thước của mảng ($1 \leq n, m \leq 10^5$).
- Dòng thứ hai chứa n số nguyên $a_1, a_2, \dots, a_n$. ($-10^9 \leq a_i \leq 10^9$)
- Dòng thứ ba chứa m số nguyên $b_1, b_2, \dots, b_m$. ($-10^9 \leq b_j \leq 10^9$)

Dữ liệu ra ghi vào tệp Bai1.out:

- In ra tổng các cặp $a_i = b_j$.

Ví dụ.

Bai1.inp	Bai1.out
8 7 1 1 3 3 3 5 8 8 1 3 3 4 5 5 5	11

Ý tưởng:

- Duyệt từ đầu đến cuối của hai mảng
- Giả sử xét vị trí thứ i của mảng $a(a[i])$, vị trí thứ j của mảng $b(b[j])$, thực hiện so sánh $a[i]$ và $b[j]$:
 - + Nếu $a[i] > b[j]$ cần tăng j lên 1 đơn vị ($j++$) để cần tìm $b[j]$ lớn hơn $b[j]$ hiện tại nằm bên phải để sao cho $a[i] = b[j]$
 - + Nếu $a[i] < b[j]$ cần tăng i lên 1 đơn vị ($i++$) để cần tìm $a[i]$ bên phải lớn hơn $a[i]$ hiện tại để sao cho $a[i] = b[j]$
 - + Nếu $a[i] = b[j]$ đếm số lượng trong mảng a , gọi $cnt1$ là số lần $b[j]$ xuất hiện trong a , tương tự đếm số lượng trong mảng b , gọi $cnt2$ là số lần $a[i]$ xuất hiện trong mảng b .
- + Cập nhật tích $cnt1 * cnt2$ vào kết quả.
- Kết của bài toán.

Code C++ tham khảo theo link: <https://ideone.com/IN91Fp>

Code Python tham khảo theo link: <https://ideone.com/anYKuj>

Bài 2. Số điểm tối thiểu

Trong một ngày, ta biết thời gian đến và đi của N chuyến xe tại bến Mỹ Đình. Hãy tìm số chỗ đỗ xe tối thiểu cần có trên sân để không có xe nào phải chờ đợi.

Dữ liệu vào từ tệp **Bai2.inp** gồm:

Dòng đầu tiên chứa T là số bộ dữ liệu, mỗi bộ dữ liệu gồm:

- Dòng đầu chứa số nguyên N
- Dòng thứ 2 chứa N số nguyên, các số cách nhau một dấu cách, trong đó số i và $A[i]$ cho biết thời điểm đến của xe thứ i
- Dòng thứ 3 chứa N số nguyên, các số cách nhau một dấu cách, trong đó số i và $B[i]$ cho biết thời điểm rời bến của xe thứ i

Thời gian điểm đến và đi biểu diễn bằng một số nguyên dương có 4 chữ số cho biết giờ (dạng 24h) và phút (60). Ví dụ: 0935, nghĩa là 9 giờ 35 phút.

Ràng buộc

- $1 \leq T \leq 100$; $1 \leq N \leq 10^4$
- $0000 < A[i] \leq B[i] \leq 2359$, $i = 1..N$

Dữ liệu ra: Kết quả ghi vào tệp **Bai2.out**

Ứng với mỗi bộ test in ra một dòng, chứa số chỗ đỗ xe tối thiểu cần bố trí trên sân để không có xe nào phải chờ đợi.

Ví dụ

Bai2.inp	Bai2.out
2	3
6	1
0900 0940 0950 1100 1500 1800	
0910 1200 1120 1130 1900 2000	
3	
0900 1100 1235	
1000 1200 1240	

Ý tưởng:**Sử dụng kỹ thuật 2 con trỏ**

- Sắp xếp hai dãy A và B tăng dần
- Khởi tạo con trỏ $i=0$ của mảng A, con trỏ $j=0$ của mảng B, $ans=1$, biến đếm $c=0$.
- Lặp khi nào ($i < n$ và $j < n$):
 - + Nếu $A[i] \leq B[j]$ thì tăng $c++$, cập nhật $ans = \max(ans, c)$, tăng $i++$
 - + Ngược lại thì giảm $c--$, tăng $j++$

Code C++ tham khảo theo link: <https://ideone.com/O7NRS8>

Code Python tham khảo theo link: <https://ideone.com/LGUCMI>

PHẦN III: KẾT LUẬN

1. Với mục tiêu đề ra đề tài đã làm được

Đề tài được trình bày trong 2 phần chính. Nêu ra thực trạng, những vấn đề cần giải quyết hiện nay mà một giáo viên bồi dưỡng học sinh giỏi cần phải chú ý. Trình bày cụ thể, đầy đủ, chi tiết về mặt lý thuyết của thuật toán hai con trỏ. Mô phỏng thuật toán bằng các hình ảnh và video cụ thể sinh động, giúp người đọc dễ nắm bắt kiến thức của thuật toán.

Ngoài ra còn lấy những bài toán cơ bản nhằm nâng cao kỹ năng, phương pháp, cũng như vận dụng vào việc lập trình giải một số bài toán một cách hiệu quả.

Đề tài đã được áp dụng vào việc bồi dưỡng, ôn thi học sinh giỏi vào 3 trường THPT Trương Dương 2, THPT Con Cuông, THPT Nghi Lộc 4. Đã đạt được một số giải cao trong việc thi học sinh giỏi tỉnh năm học 2021-2022 tổ chức vào tháng 12 năm 2021 như sau:

Ở bảng B giáo viên Đặng Văn Hảo trường THPT Con Cuông bồi dưỡng Em Hoàng Trọng Huy đạt giải ba với 16,2 điểm và Em Văn Đức Cường đạt giải nhì với 18 điểm.

Ở Bảng A giáo viên Cao Ngọc Lưu trường THPT Nghi Lộc 4 bồi dưỡng Em Trần Xuân Giáp đạt giải khuyến khích với 11,9 điểm.

2. Hướng phát triển của đề tài

Trong thời gian tới chúng tôi sẽ tập trung phát triển đề tài hơn nữa, xây dựng lại hệ thống lý thuyết một cách chặt chẽ, hợp lý hơn. Đồng thời xây dựng chỉnh sửa lại, mở rộng nội dung, dễ triển khai, thân thiện hơn. Nhằm mang lại nhiều kết quả cao trong các kì thi học sinh giỏi cũng như thi vào lớp 10 chuyên tin trường Phan Bội Châu.

Ngoài ra chia sẻ đề tài một cách rộng rãi đến các giáo viên trong và ngoài nhà trường. Giúp giáo viên bồi dưỡng có thêm lựa chọn về nội dung, phương pháp trong việc ôn tập và bồi dưỡng học sinh giỏi, thi vào lớp 10 chuyên tin trường Phan Bội Châu một cách dễ dàng và hiệu quả hơn.

3. Kiến nghị và đề xuất

Để áp dụng đề tài hiệu quả trong quá trình bồi dưỡng giáo viên cần bồi dưỡng nhiều hơn nữa về lý thuyết hai con trỏ. Xem và làm các bài toán từ cơ bản, điển hình cho đến nâng cao. Một số thao tác xử lý bằng phương pháp hai con trỏ và các phương pháp khác để ứng dụng phương pháp này cũng như kết hợp các phương pháp với phương pháp này một cách hiệu quả nhất.

Hướng dẫn, khuyến khích học sinh tham gia giải bài, tìm kiếm các bài tập có phân dạng về xử lý bằng phương pháp hai con trỏ trên các trang mạng giải bài trực tuyến

(đặc biệt trên trang <https://vn.spoj.com/>, <http://laptrinhphothong.vn/>) và các đề thi học sinh giỏi, đề thi vào trường Phan qua các năm.

Do thời gian cũng như năng lực cá nhân còn hạn chế, nên đề tài còn có một số thiếu sót, xây dựng các nội dung, phương pháp và cách thức mới chỉ đáp ứng được yêu cầu cơ bản của đề tài đưa ra. Chính vì vậy rất mong nhận được những lời góp ý, nhận xét từ Ban Giám khảo, các bạn đồng nghiệp để phát huy những mặt mạnh, điều chỉnh, khắc phục những thiếu sót cho đề tài được hoàn thiện hơn.

Lời cuối cho phép tôi xin chân thành cảm ơn ban giám khảo, ban giám hiệu nhà trường, các đồng nghiệp, các em học sinh trên địa bàn đã giúp đỡ và tạo điều kiện để tôi hoàn thành và triển khai đề tài một cách hiệu quả nhất.

TÀI LIỆU THAM KHẢO

1. Hồ Sĩ Đàm _ Tài liệu giáo khoa chuyên tin quyển 1
2. Hồ Sĩ Đàm _ Sách giáo khoa Tin học 11
3. Lê Minh Hoàng _ Chuyên đề tin
4. Đỗ Xuân Lôi _ Cấu trúc dữ liệu và giải thuật
5. Nguyễn Thanh Thủy _ Ngôn ngữ lập trình C
6. Các trang Internet
 - <http://vnoi.info/>
 - <http://vn.spoj.com>
 - <http://laptrinhphothong.vn>
 - <https://www.geeksforgeeks.org/>
 - <http://lequydon.ntucoder.net/Home>
 - <https://codeforces.com/>

PHỤ LỤC

Do hạn chế về kích thước của đề tài mà một số bài tập cũng như các video về thuật toán hai con trỏ ko đưa vào đề tài được. Vì vậy để các bạn đọc có thêm các bài tập để luyện và thêm các video mô phỏng về thuật toán nên chúng tôi đã đưa các bài tập và các video mô phỏng đầy đủ về các ví dụ cụ thể lên các trang website. Nhằm giúp người đọc có thêm nội dung cũng như dễ hiểu hơn, dễ nắm bắt hơn về thuật toán hai con trỏ.

Link tổng hợp các bài tập tương tự có hướng dẫn, có code viết trên 2 ngôn ngữ lập trình C++ và Python giúp người đọc cũng cố lại về các bài toán sử dụng thuật toán hai con trỏ một cách đầy đủ và tốt hơn.

<https://docs.google.com/document/d/1LGshYVu14bkHo4Z23YwqzgDpL4MxxzKf/edit?usp=sharing&oid=117502722904215416759&rtpof=true&sd=true>

Link các video mô phỏng thuật toán hai con trỏ qua các ví dụ cụ thể trong đề tài giúp người đọc dễ hình dung, dễ hiểu hơn.

1. Link mô phỏng thuật toán một con trỏ ở đầu và một con trỏ ở cuối di chuyển vào giữa cho đến khi cả 2 gặp nhau:
<https://youtu.be/WGz-uOsUWEo>
2. Link mô phỏng thuật toán một con trỏ di chuyển chậm và một con trỏ di chuyển với tốc độ nhanh hơn:
<https://youtu.be/5DzUbORvp4U>
3. Link mô phỏng thuật toán hai con trỏ di chuyển trên hai mảng hoặc xâu:
<https://youtu.be/GaeZ4bexOeg>

Link mô phỏng So sánh cài đặt thuật toán 2 con trỏ và một số thuật toán khác:

1. Link mô phỏng thuật toán: <https://youtu.be/39F4jKIn1OI>
2. Link mô phỏng thuật toán: <https://youtu.be/zxN2dvZxapQ>
3. Link mô phỏng thuật toán: <https://youtu.be/kVt7CK7PabA>

Mọi thắc mắc của các bạn đọc có thể liên hệ với tác giả:

1. Đinh Viết Mạnh, Giáo viên trường THPT Tương Dương 2, Huyện Tương Dương, Tỉnh Nghệ An. Số điện thoại: 0949239048
2. Đặng Văn Hào, Giáo viên trường THPT Con Cuông, Huyện Con Cuông, Tỉnh Nghệ An. Số điện thoại: 0989456448
3. Cao Ngọc Lưu, Giáo viên trường THPT Nghi Lộc 4, Huyện Nghi Lộc, Tỉnh Nghệ An. Số điện thoại: 0843637333